CleanBench: A Real-World Benchmark for Unsupervised Data Cleaning [Experiment, Analysis & Benchmark]

Kaiwen Chen*
University of Toronto

Yeye He
Microsoft Research

Tal Kariv
Microsoft

Itay Ariav
Microsoft

Yair Helman
Microsoft

Surajit Chaudhuri
Microsoft Research

ABSTRACT

Detecting and repairing errors in tables is hard, because real errors are rare, and real-world tables are full of unfamiliar but legitimate values that can resemble errors. For these reasons, building benchmarks by labeling real errors exhaustively is also hard, and existing benchmarks rely mostly on only a handful of tables (~10) that have been reused repeatedly, many with synthetically-injected errors that are not very realistic and prone to overfitting.

We present CleanBench, the first large-scale benchmark of naturally occurring errors from 1000 real-world tables. It contains a comprehensive label of 1,409 labeled erroneous cells out of 364,804 total cells, covering diverse error types such as spelling mistakes, formatting problems, and inconsistencies. We present a new and reproducible method that allows us to scale labeling, using LLM-generated candidates followed by careful human verification, with precise statistical guarantees on label quality and coverage. CleanBench enables a systematic evaluation of diverse data-cleaning techniques on real errors, which we believe is an important step toward supporting future research in this area.

PVLDB Reference Format:
Kaiwen Chen, Yeye He, Tal Kariv, Itay Ariav, Yair Helman, and Surajit Chaudhuri. CleanBench: A Real-World Benchmark for Unsupervised Data Cleaning [Experiment, Analysis & Benchmark]. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/kckevinchen/CleanBench.

1 INTRODUCTION

Data cleaning tackles the problem of detecting and repairing errors in tables. Prior studies report that 24% of professionally produced spreadsheets contain errors [54], and roughly 20% of the data tables that accompany scientific publications suffer from quality problems [71]. These issues carry serious financial and societal costs, as evidenced by a steadily growing collection of "horror stories" [25].

The importance of data cleaning has motivated a long and fruitful line of academic research [11, 17, 18, 20–24, 26, 28–30, 33, 35, 38, 45, 47, 49, 55, 59–62, 64, 65, 67]. This research has also influenced commercial systems, with automated data cleaning features now available to the masses through commercial offerings, such as Microsoft Excel [50] and Google Sheets [32].

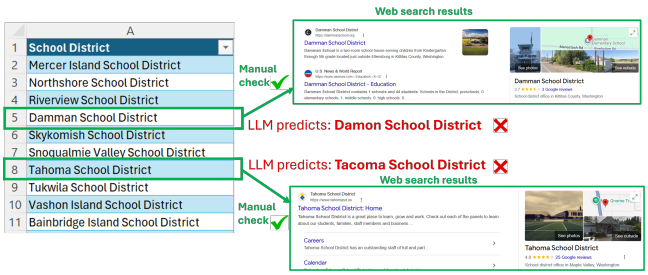


Figure 1: Challenging example for data cleaning

Finding errors is hard for humans and algorithms. Despite decades of research, finding errors in tables continues to be hard, for two main reasons. First, **(C1)** errors are rare in real tables, occurring in only a small fraction of cells in an ocean of table content, creating a "needle in a haystack" challenge. Second, **(C2)** tables often contain unfamiliar content, with proprietary, domain-specific content and uncommon values (e.g., abbreviations and code names), that appear rarely elsewhere and can therefore resemble errors. Humans as well as algorithms can struggle to distinguish true errors from rare but correct values in such cases.

Figure 1 shows a real example column we encounter. This column lists names of hundreds of school districts. It is hard for humans or algorithms to identify errors reliably in tables like this, because some of these names can appear rare/uncommon. In fact, even today's frontier LLM can frequently make mistakes – e.g., it predicts the uncommon "`Dammon School District`" as the more common "`Damon`", and "`Tahoma`" as "`Tacoma`", etc., which, when cross-validated against external evidence (web searches on the right), would be revealed as false positives. Note that these are precisely the kinds of predictions real data cleaning systems need to make, in places like Microsoft Excel Clean Data [50] and Google Sheets Smart Cleanup [32].

It is known in the literature that achieving both high precision and high recall in data cleaning is difficult on diverse tables [7, 65]. Achieving high precision is hard because of the rarity of errors **(C1)**, causing a severe class imbalance between clean/erroneous cells, with the vast majority of cells (say 99%) containing no errors. Consider an algorithm A with perfect recall that catches every true error (1% of cells). Even if A has a very low false-positive rate of

*Work done while at Microsoft

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Table 1: Commonly used data-cleaning benchmarks. Each row describes one benchmark dataset with exactly one table.

Dataset	Introduced by	(Rows, Cols.)	Cells with labels	Cells labeled as errors	Cell error rate	Labels obtained by	Used by
Real errors							
Flights	[59]	(2,376, 7)	16,632	4,920	29.58%	Merged inconsistent data from different sources	[8, 17, 44, 46–48, 52, 53, 59, 66]
Movies	[48]	(7,390, 17)	125,630	7,675	6.11%	Merged inconsistent data from different sources	[8, 17, 44, 46, 48, 53, 66]
Beers	[48]	(2,410, 11)	26,510	4,362	16.45%	Manually labeled	[8, 17, 44, 46–48, 52, 53, 66]
Rayyan	[7]	(1,000, 11)	11,000	948	8.62%	Manually labeled 1,000 rows	[7, 8, 17, 44, 46–48, 52, 53, 66]
Food	[34]	(170,945, 15)	45,000	1,208	2.68%	Manually labeled 3,000 rows	[17, 34]
Synthetic errors							
Hospital	[20]	(1,000, 20)	20,000	509	2.55%	Injected typos by inserting many "x" characters	[8, 34, 48, 52, 59, 66]
Tax	[10]	(200,000, 15)	3,000,000	121,219	4.04%	Synthetically injected using BART [10]	[10, 17, 46–48, 52, 53, 66]
Soccer	[58]	(200,000, 10)	2,000,000	31,296	1.56%	Synthetically injected using BART [10]	[17, 34, 44, 46]
Adult	[58]	(97,684, 11)	1,074,524	1,062	0.10%	Synthetically injected using BART [10]	[17, 24, 34]

1% on the clean cells, its precision drops to roughly only 50% ($\approx \frac{1\%}{1\%+1\%}$), a level unacceptably low for many applications.

Conversely, algorithm A can improve precision by being more conservative and abstaining on unfamiliar or uncertain values (e.g., the uncommon names in Figure 1), but this significantly lowers recall, since tables contain a large amount of unfamiliar content, as discussed in **(C2)**.

Existing benchmarks are small, often with synthetic errors. Like in any predictive problem, good benchmarks are essential for measuring progress and comparing algorithms. For a good data cleaning benchmark, it not only needs to have **(R1)** high accuracy, meaning every labeled error must be a true error, which is to be expected; it must also have **(R2)** high coverage, or *all* errors present in T must be comprehensively labeled, with few or no errors left unlabeled in benchmark tables.

While ensuring **(R1)** high accuracy is relatively easy as humans can manually verify every cell labeled as an error, ensuring **(R2)** high coverage is much more difficult, as humans must check every cell in the entire dataset for missed error labels, or otherwise unlabeled errors can unfairly penalize perfectly valid predictions from algorithms being benchmarked. Consider a moderately sized benchmark of 100 tables – even if each table has only 100 rows and 10 columns, that amounts to a total of 100K cells to manually check for missed errors. Given the previously discussed **(C2)** that many of these cells contain unfamiliar table content, it becomes prohibitively expensive for humans to verify quickly, making **(R2)** high coverage really hard to ensure in practice even at a small scale.

Because of the challenges in ensuring high coverage at scale, the data cleaning literature has historically relied on a small number (~10) of benchmark tables. Table 1 summarizes commonly-used benchmark tables in the literature, where many have synthetically-injected errors rather than naturally occurring ones (to be reviewed in detail in Section 2).

While these benchmarks are clearly useful in the absence of better alternatives, they carry several important risks, such as: (1) over-fitting to a handful of tables, when only ~10 tables are used; (2) reliance on less realistic errors, often produced by hand-written rules that are prone to over-fitting (e.g., errors in the `Hospital` dataset were created synthetically by inserting many random "x" characters into table cells [48, 59], which is not entirely realistic and easy for ML algorithms to over-fit); and (3) over-representation of certain errors, when the same type of errors are injected repeatedly in the same table (e.g., the `Hospital` dataset contains only one type of error, which is the randomly-inserted "x").

CleanBench: A first large-scale benchmark with real errors in real tables and quality guarantees. We aim to substantially change the landscape of benchmarking for data cleaning, by developing large-scale benchmarks with real, naturally-occurring errors.

In this work, we present CleanBench, a benchmark built by comprehensively labeling 1,000 real tables, with 500 relational tables and 500 spreadsheet tables. Of the 364,804 total table cells in these tables, 1,409 are labeled as errors that occur in 168 tables. This gives a cell-level error rate of 0.4% and a table-level error rate of 16.8%, in line with past estimates of error rates in real tables [54, 71].

To reach this scale, it is prohibitively expensive to manually label each and every table cell (to ensure no missed errors). Our key insight is that today's frontier LLMs have advanced to the point where, by querying them repeatedly on every table, and by targeting specific types of errors separately (e.g., misspellings, formatting, etc.), LLMs can exhaustively flag every potentially erroneous cell as a candidate for human inspection, with few or no missed errors, which we validate both statistically and empirically (Section 3).

Although LLMs still cannot reliably distinguish subtle cases (e.g., uncommon values vs. genuine errors like in Figure 1), once they are reliable enough to flag all potential errors for human inspection after repeated queries, that can already drastically reduce the amount of manual labeling work by orders of magnitude, given the sparsity of real errors, making it finally feasible to label at the scale of 1,000 tables.

We model the repeated LLM querying process using statistical models, such as those for species estimation [14], so that we can stop repeated queries adaptively on each table at a suitable time in a principled manner, with precise probabilistic guarantees of not missing any unknown errors in each table.

By combining principled LLM querying to flag all possible errors, with extensive manual verification, we argue that it is finally possible to create a large-scale benchmark, with real errors exhaustively labeled in real tables. We create the first such benchmark, CleanBench, to systematically evaluate existing data cleaning algorithms.

Contributions. We make the following contributions:

- We built the first large-scale benchmark with naturally occurring real errors in real tables, greatly improving resources available in research to evaluate data cleaning algorithms.
- We design principled methods that combine repeated LLM queries with statistical modeling, and human labeling, to finally make it feasible to label real errors at scale, with precise quality guarantees.
- Our new benchmark allows us to systematically evaluate the performance of existing data cleaning methods on real-world tables, including academic algorithms, commercial systems (e.g., Microsoft Excel and Google Sheets), as well as LLM-based methods using frontier models. We present an extensive experimental evaluation across 20+ representative methods.

CleanBench data, as well as the labeling system we built to facilitate labeling tabular data, are available for future research.

2 RELATED WORK

Exact data cleaning for data analytics vs. Statistical cleaning for ML. Depending on target applications, there are two main lines of existing work in data cleaning, where one focuses on fixing each and every erroneous cell in a table, with downstream applications in data analytics (where all data values are expected to be correct in an exact sense) [17–23, 26, 29, 35, 38, 57, 59, 60, 65, 67], while the other focuses on cleaning data for ML applications, where statistical utility (e.g., improved downstream ML models) is measured over exact cell-by-cell correctness [39, 40, 42].

In this work, we focus on the "exact data cleaning" scenario for data analytics, which is important in many practical applications such as spreadsheet software (Excel and Google Sheets). We measure exact cell-by-cell correctness, which, unlike statistical cleaning, can be measured independently of downstream ML modeling.

Existing data cleaning benchmarks. Data cleaning is a long-standing challenge in data management, but given the difficulty in comprehensively labeling all errors in a table, existing work relies on a small number of benchmark tables.

Table 1 lists 9 benchmarks (each with 1 table only), frequently used in existing data cleaning research, which we review below:

Tables with "real errors" (some on synthetically generated tables). While the first 5 benchmark datasets in the table fall in the category of "real errors", some of the underlying data is synthetically constructed (as opposed to naturally occurring), to intentionally create errors. For example, the "`Flights`" dataset shown in the first row is constructed by merging flight information from 38 different sources [43], to intentionally create inconsistency (e.g., source-1 has the arrival time of a flight as "`10:30 am`", while source-2 may record the same as "`10:31 am`", thereby creating inconsistency). Although labels can then be constructed this way automatically, by using one underlying source as the ground-truth, this creates an exceedingly large number of inconsistencies (present in 29% of cells), with no other types of errors beyond inconsistency. Other benchmarks, such as "`Movies`" in the second row of the table, are constructed similarly.

Other datasets in this category, such as "`Rayyan`" and "`Food`", are manually labeled using select sub-samples of rows (in order to reduce the manual labeling effort required to identify all possible errors, as discussed earlier).

Tables with "synthetic errors". The bottom half of Table 1 shows 4 commonly-used datasets, with synthetically-injected errors based on deterministic rules, so that no manual labeling is required and labels can be automatically generated. Because of the synthetic nature of the injected errors, they may not always realistically represent naturally occurring errors – for instance, the "`Hospital`" dataset injects synthetic spelling errors by repeatedly inserting the same letter "`x`" into many different cells [48, 59], making such issues easy for algorithms to detect and overfit to (in fact, even a simple rule that flags any cell with "`x`" as an error can easily "over-fit" on this dataset, with 99.8% precision).

The use of a small number of benchmark tables, together with synthetically-injected errors or synthetically-constructed datasets, creates the risk of algorithms overfitting on a small set of errors that are not entirely representative of real and naturally-occurring errors, making it hard to judge the real-world effectiveness of different algorithms.

Partially-labeled benchmarks via "pooling". Prior work also attempts to benchmark a broader collection of tables. Auto-Detect [35] and later MaTElDa [8] used "pooling" from labeling IR ranking [72], to "pool" or union top-N error predictions from a set of K algorithms being evaluated on real tables, and then only label the union of this subset of top predictions.

While "pooling" allows evaluations beyond a small number of tables, its labels are tied to a specific set of K algorithms, and cannot be reused to evaluate new algorithms (as new predictions not previously labeled in a previous "pool" can be produced), which therefore does not generate reusable benchmarks to continuously evaluate progress of new data cleaning research.

Benchmarks for missing cell prediction. Predicting values for missing cells in tables is a closely-related task [45, 69], which we do not consider in CleanBench, as it is often easy to generate realistic benchmarks for missing-cell prediction (e.g., by masking random cells in real tables and using the masked cell values as the ground-truth for predictions [45]).

Existing data cleaning methods. We give a brief overview of representative classes of methods, in the area of unsupervised data cleaning. We refer readers to additional surveys such as [19, 36, 57] for a more comprehensive review of additional methods, such as data cleaning requiring user-provided labels or other forms of input.

Constraint-based methods. There is a large body of methods using constraints, such as functional-dependencies or denial constraints, to perform data cleaning [11, 18, 20, 22, 26, 30, 38, 59, 60]. While such methods may be effective on select benchmarks (e.g., in Table 1), we find in our large-scale evaluation that these methods may have limited effectiveness on naturally-occurring errors in real tables, where existing forms of constraints have limited coverage.

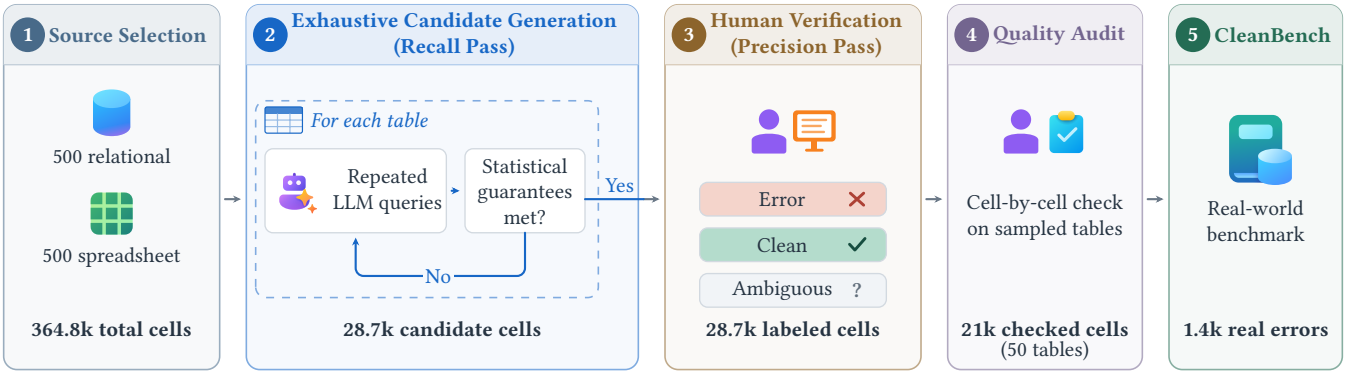


Figure 2: Overview of CleanBench construction

Commercial systems. Given the importance of automated data cleaning in tabular data, commercial systems like Microsoft Excel [50] and Google Sheets [32] have recently added features to automatically clean data for spreadsheet users. Such features scan user tables and can generate proactive suggestions (as suggestion cards on the side pane) for users to review and accept, thereby democratizing data cleaning for the masses.

LLM-based methods. LLMs are shown to be effective on diverse tasks, including table-oriented tasks such as data cleaning [13, 41]. Rapid advances in LLM capabilities in recent years have made these models strong contenders for automated data cleaning. We explore their effectiveness in our experimental evaluation using CleanBench.

Machine Learning (ML)-based methods. Data cleaning methods based on traditional Machine Learning (ML) typically require labeled training data for *each individual table* [34, 47, 49, 51, 68]. We consider such methods currently out of scope for evaluation using CleanBench, as real errors are rare in real tables (we observe a mean of only 1.41 errors per table, in tables averaging 364 cells), such that asking users to identify sparse errors and provide them as training labels can become challenging if not impractical in such settings.

3 CLEANBENCH: CONSTRUCTION

We will first give an overview of the benchmark construction process, before describing each step in detail in this section.

3.1 Overview

Figure 2 gives an overview of the main steps in our benchmark construction process, which systematically combines repeated LLM queries and human verification, to scale labeling to 1000 real tables with precise quality guarantees.

In step (1) Source selection, we first randomly sample 500 real relational tables and 500 spreadsheet tables from public sources to ensure data tables used in the benchmark are diverse and representative, and that they contain real errors in the wild. There are a total of 364.8k cells from these 1000 tables. (Section 3.2).

Next, in step (2) Exhaustive candidate generation (recall pass), we leverage the observation that while today's LLMs still cannot fully understand the nuance of tables and be 100% accurate in detecting errors, they are already very good at exhaustively *flagging all cells that are potentially erroneous*, when we repeatedly query LLMs on the same table T until statistical convergence (we perform principled statistical reasoning adaptively on each T, and stop repeated queries only when the chance of missing any unknown errors in T is diminishingly small).

Table 2: Statistics across benchmark construction steps.

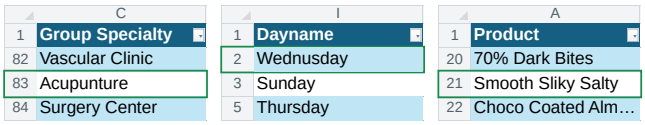
Statistic	Total	Relational	Spreadsheet
(1): Total input tables	1,000	500	500
(1): Total input cells	364,804	142,459	222,345
(2): Candidate tables flagged for review	697	259	438
(2): Candidate cells flagged for review	28,745	7,184	21,561
(3): Tables labeled to contain errors	168	44	124
(3): Cells labeled as errors	1,409	347	1,062
(3): Error / per cell	0.0039	0.0024	0.0048
(3): Error / per table	1.41	0.69	2.12

Because this is set up as a recall-oriented step, we refer to it as the "recall pass", where the goal is to not miss any potentially erroneous cells (as outlined in **R2** of the introduction). This LLM-assisted exhaustive candidate generation step reduces the 364.8k original table cells significantly to 28.7k cells after this pass, making it possible to manually verify only these candidate cells, with only a small bounded probability of false negatives that we validate both theoretically and empirically (Section 3.3).

In step (3) Human verification, which we also refer to as the "precision pass", human labelers carefully inspect each of the 28.7k candidate cells that are potentially erroneous, using a labeling system we develop (Figure 7) that combines the original table context with multiple pieces of external evidence (e.g., web search results like in Figure 1), to reach a final labeling decision (Section 3.4).

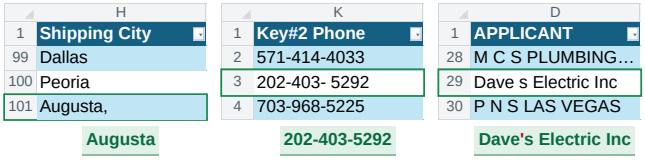
Finally, in (4) Quality audit, we perform an extensive review of the annotated labels, by sampling tables and having reviewers independently inspect each and every cell exhaustively in these tables (without leveraging LLMs), to cross-check label quality and verify that no or very few errors are missed (Section 3.5).

Types of errors. We now give an overview of error types used in our benchmark, as we additionally tag each instance of cell error with an "error type" to enable additional downstream analysis



(a) **missed letter** (b) **replaced letter** (c) **transposed letter**

Figure 3: Real examples of spelling errors in CleanBench



(a) **Trailing comma** (b) **Phone spacing** (c) **Apostrophe**

Figure 4: Real examples of formatting errors in CleanBench

of benchmark results. We use three broad error types: spelling, formatting, and inconsistency, described below.[1] [2]

Spelling. Spelling errors are an intuitive type of error, where cell values are spelled incorrectly in a table context. Figure 3 shows examples of real spelling errors we encountered in CleanBench, with missed, replaced, and transposed letters, respectively.

Note that because tables can often carry unfamiliar or domain-specific content, a determination of misspelled vs. correct values is not always trivial and may require additional evidence from external sources, as shown in Example 1.

Formatting. A formatting error is another intuitive type of error, due to violations of formatting conventions in punctuation or spacing. Figure 4 shows examples of real formatting issues in CleanBench, with each involving different punctuation.

Inconsistency. Inconsistency is a broad type of error, covering cell values in a table that are inconsistent in (1) single-column, (2) cross-column, and (3) cross-table context.

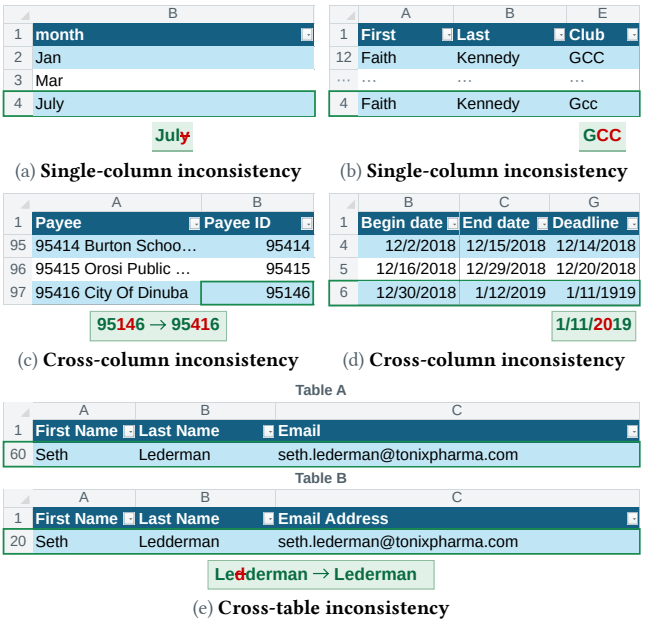
(1) Single-column inconsistency arises when a value in a cell is inconsistent with other values in the same column, as shown in Figure 5(a) where "`July`" is inconsistent with all remaining values in the column that use 3-letter month abbreviations, and in Figure 5(b) where a single "`Gcc`" is inconsistent with many other upper-cased "`GCC`" in the same column.

Single-column inconsistency also includes cases involving numerical outliers (abnormally large or small values in a column), or misaligned column values (e.g. with categorical values mixed in a numerical column due to parsing errors), etc.

(2) Cross-column inconsistency arises when a cell value is inconsistent with values in other columns, as shown in Figure 5(c), where the first part of column "`Payee`" is supposed to match values in "`Payee ID`", which, however, is violated in the last row; and in Figure 5(d), where the value "`1/11/1919`" is anomalous and

[1]Prior studies also propose more fine-grained taxonomies for errors, such as classifying errors based on the cause of errors (e.g., OCR scan errors vs. typo errors) [12]. In the absence of such information for real errors in real tables, we adopt a coarse-grained classification for error types that are less subjective and easier to ascertain.

[2]We note that the three error types are not always mutually exclusive, as a misspelled name in a cell can sometimes cause additional cross-column inconsistency, so a cell may be tagged with more than one error-type.



(a) **Single-column inconsistency** (b) **Single-column inconsistency**

(c) **Cross-column inconsistency** (d) **Cross-column inconsistency**

(e) **Cross-table inconsistency**

Figure 5: Real examples of inconsistency errors

violates relationships implicit in the table (e.g., "`Deadline`" has to be between "`Begin date`" and "`End date`"). Traditional multi-column constraints such as Functional Dependency and Denial Constraints are usually in this category.[3]

(3) Cross-table inconsistency arises when a cell value in one table becomes inconsistent with values in another table from the same source (e.g., spreadsheet tables in the same "`.xlsx`" file or relational tables in the same databases are considered "same source").

Figure 5(e) shows such a real example in our benchmark, where the relevant row appears in two separate spreadsheet tables, from within the same file, with matching "`email`" and "`first-name`", but with "`last-names`" that differ by one character. We holistically check all tables from within the same source for possible cross-table inconsistencies.

We note that single-column / cross-column / cross-table inconsistencies are not meant to be mutually exclusive, as one erroneous cell can manifest itself as violating more than one type of inconsistency at the same time (e.g., both single-column and cross-column like in the case of Figure 5(d)). We therefore group all forms of inconsistencies in one category, without breaking them down as separate categories.

3.2 Source Selection

We are now ready to explain how CleanBench is built step-by-step.

To ensure CleanBench represents real tables and naturally occurring errors in the wild, in step (1) source selection shown in Figure 2, we sample two types of tables from web sources: (1) 500 relational tables extracted from Business Intelligence models ("`.pbix`" files), and (2) 500 spreadsheet tables from spreadsheet workbooks

[3]Some cross-column inconsistencies, such as the one in Figure 5(c), are more subtle and cannot be captured by traditional constraints.

(".xlsx" files), with similar tables de-duplicated to maintain diversity.[4]

We sample from tables in English that do not contain an exceedingly high number of empty cells (tables with more than 30% empty cells are excluded, as we find their table semantics often become unclear). In order to prevent skewed representation from large tables, and to make careful manual inspection of entire tables possible, we sample the first 100 rows from each table for labeling.

We preserve the original table source metadata to ensure that multi-table errors can also be identified (e.g., Figure 5(e)), by linking tables from the same source (e.g., spreadsheet tables from the same workbook file, or relational tables from the same database).

Stage output. A total of 1000 tables are selected to move to the next stage, with 364.8k cells. Table 2 reports detailed statistics of this and other steps in the construction process.

3.3 Exhaustive Candidate Generation

Next, using the 1000 selected real tables, we reach step (2) of Figure 2, where we leverage LLMs to exhaustively flag all candidate cells that may be erroneous, based on the observation that while today's LLMs still cannot reliably tell subtle differences between uncommon values and erroneous values in table contexts, to be 100% correct in predicting errors (e.g., Figure 1), they are already very good at exhaustively catching all candidate cells that may be erroneous, especially when we repeatedly query LLMs on the same table T until statistical convergence, which we will explain in detail in this section.

Because this phase is set up as a recall-oriented step, where the goal is for LLMs to catch all possible errors to ensure close to full recall, we refer to this also as the "recall pass".

LLM queries for candidate errors. We use LLMs by explicitly instructing models to flag all plausible cells that may have errors (together with an explanation of the suspected error and a suggested correction), as candidates for additional human reviews.

To ensure that models can be exhaustive, we prompt LLMs to focus on one specific type of error at a time, described below:

Spelling errors. For spelling errors like in Figure 3, we query LLMs to identify all cells that may be incorrectly spelled, encouraging the model to flag all suspicious cells even if it is unsure about them, including cases in Figure 1 that may turn out to be false-positives.

Formatting errors. For formatting errors, we give examples of formatting issues like in Figure 4 as few-shot examples, and ask LLMs to identify any formatting issues involving punctuation or whitespace, including but not limited to the given examples.

Inconsistency errors. Since inconsistency covers different forms of errors that manifest as (1) single-column, (2) cross-column, and (3) cross-table errors, as explained in Section 3.1, we give the LLM separate prompts to check each form of inconsistency, so that it can focus on a narrow category of issues and be more comprehensive in its discoveries.

Example prompts for spelling, formatting, and the three forms of inconsistency appear in Appendix A.

Repeated queries with statistical guarantees. Because LLM outputs are stochastic, a single query on a table T can miss possible errors. For each error type, we therefore perform repeated LLM queries on T adaptively, and only stop when a form of statistical convergence is reached. This lets the number of LLM queries vary with the complexity of T, while providing a statistical guarantee of coverage, which we describe below.

Specifically, we model an LLM's discovery of possible error cells in a table T, as analogous to biologists discovering new species in the wild, where statistical processes such as species estimation models [63] can naturally apply. Recall that in species estimation, biologists like Darwin would collect samples repeatedly (e.g., traps collected over many days), and use the rate of new discoveries (species never seen before) versus re-discoveries (species already discovered) over multiple samples, to estimate how many unseen species still remain if sampling were to continue indefinitely.

This is analogous to LLM discovery of possible errors in a table – intuitively, if repeated LLM queries are continuously rediscovering the same set of candidate cells, with no new/unseen discoveries, then the chance of seeing new discoveries will keep shrinking, until we can stop safely at a point when the probability of missing new discoveries is provably small. On the other hand, if repeated LLM queries keep surfacing new candidate cells not discovered in previous queries, then the queries should continue until saturation.

More specifically, we use two forms of statistical estimators from the species estimation literature, (1) Good–Turing estimator [31] that estimates the relative probability mass of unseen candidates, and (2) Chao1 estimator [15] that estimates the total number of unseen candidates in absolute terms.

Let S_t denote the number of distinct candidate cells discovered after t queries, and let $f_{t,j}$ denote the number of distinct cells observed in exactly j queries. Thus, $S_t = \sum_{j\geq 1} f_{t,j}$, whereas $N_t = \sum_{j\geq 1} j f_{t,j}$ counts candidate occurrences across queries, including rediscoveries. Each cell is counted at most once within a query.

We compute the Good–Turing-based score [31] using the candidates observed only once to estimate the unseen probability mass, yielding the estimation:

$$\hat{p}_{0,t} = f_{t,1} / \max(1, N_t). \tag{1}$$

Chao1 [15] instead uses the frequencies of candidates observed once and twice to estimate the total number of candidates:

$$\hat{S}_t = S_t + \begin{cases} f_{t,1}^2/(2f_{t,2}), & f_{t,2} > 0, \\ f_{t,1}(f_{t,1}-1)/2, & f_{t,2} = 0. \end{cases} \tag{2}$$

The difference $\hat{S}_t - S_t$ provides a diagnostic estimate of remaining candidate richness. Let U_t denote the upper endpoint of the approximate 95% log-transformed Chao1 interval computed using [16]. We define the relative Chao1-based score as

$$g_t = \frac{\max(0, U_t - S_t)}{\max(1, S_t)}. \tag{3}$$

This score incorporates the interval's uncertainty adjustment relative to the number of observed candidates. We use it as a stopping diagnostic on the number of undiscovered true errors.

Algorithm 1 shows how these estimates combine operationally. It queries LLMs on each table T and stops adaptively with guarantees of statistical coverage. We start with a maximum query budget

[4] Details of the source data and its preprocessing can be found in [3].

Algorithm 1: Adaptive LLM Queries for Candidate Gen

input : Target table T; max query budget B; thresholds $\epsilon_{\text{GT}}, \epsilon_{\text{Chao}}$
output : Candidate cell locations for manual reviews

1 **for** $t = 1, \ldots, B$ **do**
2 $\quad S_t, N_t, f_{t,1}, f_{t,2}, U_t \leftarrow \text{LLM}(T)$ // *Update from the latest query*
3 $\quad \hat{p}_{0,t} \leftarrow f_{t,1} / \max(1, N_t)$ // *Good-Turing for unseen probability*
4 $\quad g_t \leftarrow \max(0, U_t - S_t) / \max(1, S_t)$ // *Relative Chao1 bound.*
5 $\quad$ **if** *check-stop*$(\hat{p}_{0,t},\ g_t,\ \epsilon_{\text{GT}},\ \epsilon_{\text{Chao}})$ **then**
6 $\quad\quad$ **break** // *We can safely stop querying*
7 **return** *all flagged cell locations*

of $B = 20$ queries for each error type on each table T. At Line 2, we perform a new LLM query and update all statistics based on the latest query discoveries on T. At Line 3 and Line 4 we compute the corresponding Good-Turing and Chao1 estimates, using Equations (1) and (3), respectively, and check if it is safe to stop querying at Line 5, by checking $\hat{p}_{0,t} \leq \epsilon_{\text{GT}}$ and $g_t \leq \epsilon_{\text{Chao}}$, with both ϵ_{GT} and ϵ_{Chao} set at the 0.05 levels.[5]

We use the following proposition, to show that Algorithm 1 provides statistical guarantees for covering all candidate errors.

Proposition 1. Given a table T, assume each LLM query independently draws its flagged candidates from a fixed underlying distribution over T. When Algorithm 1 terminates at Line 5, the probability of missing unseen errors that could be discovered by future queries is low, defined as:

(i) *(Bounded unseen probability mass)*: the probability that the next query flags a cell not already discovered is at most ϵ_{GT}.

(ii) *(Bounded missed errors)*: any given true error is missed with probability at most ϵ_{Chao}.

□

Since we set both ϵ_{GT} and ϵ_{Chao} to small numbers like 0.05, this provides statistical guarantees that we can discover all candidate errors with limited queries, among all candidates that LLM can discover, with high probability.

Also note that for some complex tables T, repeated LLM queries can use all B before satisfying the stopping condition, and become *budget-exhausted*. In such cases, we exhaustively review the entire T manually (on top of LLM discoveries), to ensure comprehensive label coverage. We note that in practice, only 1.2% of cases become budget-exhausted and require manual review.

Example 1. Figure 6 shows an example sequence of LLM queries for the school names in Figure 1. Query 1 flags "`Damman`" and "`Skykomish`", and queries 2 and 3 additionally flag "`Tahoma`" and "`Snoqualmie Valley`", resulting in four total distinct candidates. Later, Queries 4 and 5 re-discover these items in different combinations.

We calculated the Good–Turing and Chao1 estimates as shown in the figure. With the thresholds $\epsilon_{\text{GT}} = \epsilon_{\text{Chao}} = 0.05$ from Algorithm 1, neither condition is met from Query 1 to Query 4. Query 5 rediscovers the final candidate that had been seen only once ("`Snoqualmie Valley`"), at which point both estimators drop to zero in this simple example, allowing us to stop querying.

[5] At Line 5, we additionally require a minimum number of $t_{min} = 4$ initial queries to pass before allowing early stop, and stopping-conditions to hold for $K = 2$ queries before committing stop, which are safeguards above and beyond Good-Turing/Chao1.

● New candidate ○ Rediscovery

School district	Query 1	Query 2	Query 3	Query 4	Query 5
Damman	●	○	○	○	–
Tahoma	–	●	–	○	–
Skykomish	●	–	○	–	○
Snoqualmie Valley	–	–	●	–	○
Merged candidates	2	3	4	4	4
Good–Turing	1.00	0.50	0.29	0.11	**0.00**
Chao1 (U–S)/S	6.89	7.38	5.53	1.18	**0.00**
Decision	More	More	More	More	**STOP**

Figure 6: An illustrative run of Algorithm 1

Human verification will later only need to check these 4 flagged items as opposed to all hundreds of school names in Figure 1. □

Stage output. From 364.8k original table cells in 1000 tables, after this stage of repeated LLM queries, 28,745 unique cells are flagged as candidates across all LLM runs (7.8% of all unique cells), which is a significant reduction that makes manual checking feasible, precisely because real errors are sparsely populated in real tables (despite us being very conservative and recall-oriented in LLM candidate generation). Table 2 reports detailed statistics of this step, including a breakdown between relational vs. spreadsheet tables.

3.4 Human Verification

From the 28.7k unique cells exhaustively flagged as potentially erroneous, we reach step ③ of Figure 2, where we perform manual verification of each cell, to validate (1) whether a flagged cell is a true error; and (2) what possible repairs exist for each error cell.

Labeling process. Four experts, who are researchers and practitioners experienced in data cleaning, participated in the human verification stage. Two members assigned initial labels, followed by all four members conducting extensive reviews on cases requiring further discussion, until final labels were assigned for all candidates.

To make it easy to review and label each case with full context (e.g., surrounding table content, verification via web search results, etc.), we develop a labeling system as shown in Figure 7, which displays cells flagged as suspicious in their full surrounding table context, alongside an LLM-produced explanation of why this is an error, as well as its suggested fix (collected during the previous LLM querying step). The system also integrates web search.

In cases of doubt, human labelers can click on the "Verify on web" button to directly see search results (without needing to copy-/paste the values in question and their surrounding context to a search engine).

Labelers' verdicts can then be recorded as TP (true positives), FP (false positives), etc., with additional categories for uncertain cases. These cases were later resolved through discussions among the four reviewers until agreement was reached.

Labeling results. The labeled results are not just a simple black-or-white yes/no verdict – each cell receives one of 3 possible labels:

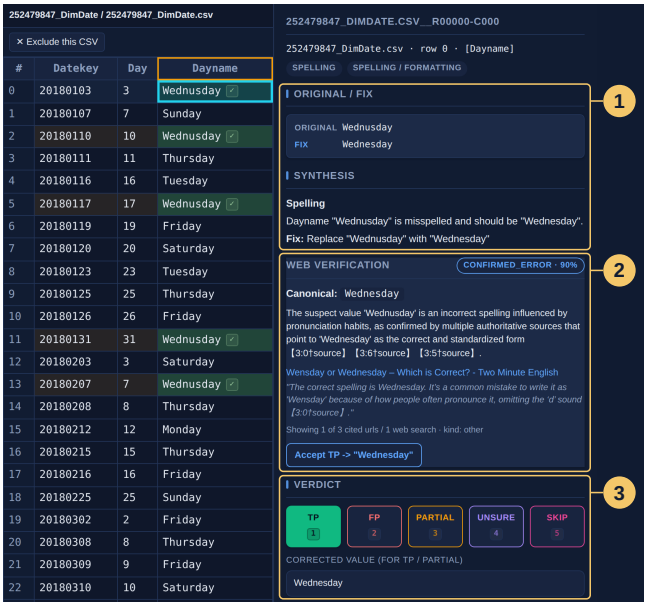


Figure 7: Labeling system for human verification: (1) error explanation and suggested repair; (2) web verification; (3) human verdict and corrected value.

- (1) *Error*: when there is convincing evidence (internally within the table or externally through web search), to establish the cell as an error;
- (2) *Clean*: when the flagged cell is established to be valid;
- (3) *Ambiguous*: when a cell is reasonably suspicious, but evidence is insufficient to convincingly establish that it must be an error, or disagreement remains after reviewer discussions.

We note that for real tables in the wild that contain niche content that humans are unfamiliar with, an "ambiguous" label category is necessary. Figure 8 shows two example cases that are judged as ambiguous: in Figure 8(a), while almost all cells in column E contain "`project-ID`" values that follow a specific regex pattern (a number followed by a dash followed by an alphanumeric string), row-60 is a lone exception whose `project-ID` is "`10551047`" that looks very different from all other values in the column. While it is plausible that this value is incorrect, there is no convincing evidence to establish that it must be wrong (web search reveals no information as these projects may be niche local projects with no web presence). We therefore label this case "ambiguous" so that an algorithm is neither overly penalized, nor unduly rewarded, for flagging this cell.

Figure 8(b) shows another ambiguous example, where "`Glove Ju!ce`" (with an exclamation point in the middle) looks like a plausible typo, though it could equally be intentional styling, common for team names. Web verification surfaces no evidence to confirm or rule out either possibility (as the team name may be too local to be indexed), so we label this case "ambiguous" as well.

Multiple possible repairs. For cases that are judged as "error", we annotate acceptable repairs, allowing more than one repair for a cell. For example, in the case of Figure 5(b), it may be plausible to repair either `GCC` to `Gcc` or the other way around. We record both as acceptable repairs, since both may be admitted as valid repairs.

(a) Suspicious project ID

	C	E
1	Project Name	project ID
50	California State Rep…	1-1878BD
52	Data Conversion	1-18728B
…	…	…
60	CA Report	10551047

(b) Suspicious team name

	B
1	Team
95	Sunshine Boys
96	Delt Thunder VIII
…	…
98	Glove Ju!ce

Figure 8: Example cases labeled as ambiguous

Stage output. From the 28.7k unique cells that require human review from the previous stage, this extensive manual labeling step produces final labels, where 1,409 cells are identified as true errors, 842 as ambiguous, and the remainder as clean. Table 2 gives a more detailed breakdown of the statistics in this step.

3.5 Final Quality Audit

Finally, in step (4) of Figure 2, we take all labels from the previous step and perform a final quality audit to independently verify the accuracy and coverage of existing labels, by manually inspecting labels from tables sampled with stratification by error count.

Specifically, two reviewers sampled 50 tables (with a total of 20,990 cells) from the labeled benchmark, and labeled these tables cell-by-cell manually (with no LLM assistance), in order to compare and cross-check labels produced via entirely manual methods, vs. ones produced with LLM-assisted pruning (Section 3.3 and Section 3.4).

The 50 sampled tables originally contain 273 cells labeled as errors and 110 labeled as ambiguous, when using LLM-assisted pruning. Both reviewers agreed with the benchmark labels in 380 of these 383 cases[6], showing a high degree of agreement (99.22%).

In addition, one reviewer flagged an additional cell as an error in the exhaustive cell-by-cell review, while the other considered it clean. This empirically validates the comprehensiveness of error coverage, even when LLM-pruning is used.

4 CLEANBENCH: BENCHMARK ANALYSIS

Using the labeled CleanBench, we then study the characteristics and distributions of real errors in the wild.

Real errors are sparse, and concentrated in few tables. Figure 9(a) shows the distribution of errors across tables, which suggests that real errors are concentrated in a small fraction of tables, with most remaining tables containing no errors – 91.2% and 75.2% of relational and spreadsheet tables have no errors, respectively. Overall, the 1,409 labeled errors occur in just 168 of the 1,000 tables.

In general, real errors are sparsely populated at both the cell and table levels, with 0.0024 / 0.0048 errors per cell for relational and spreadsheet tables, respectively; and 0.69 / 2.12 errors per table for the two, respectively, as reported in Table 2. Relational tables appear to be noticeably cleaner than spreadsheet tables, as these statistics would suggest.

Distributions of error types can vary between sources. Figure 9(b) breaks down cell-level errors by error types across the two

[6]In the remaining three cases, the reviewers disagreed with each other: one reviewer labeled them as errors, while the other considered them ambiguous.

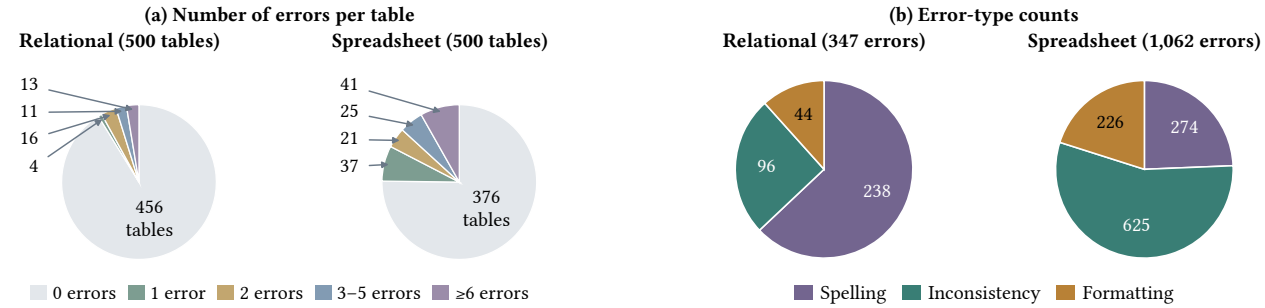


Figure 9: Error distributions across (a) tables and (b) error types. A small fraction of cells can have more than one type of error.

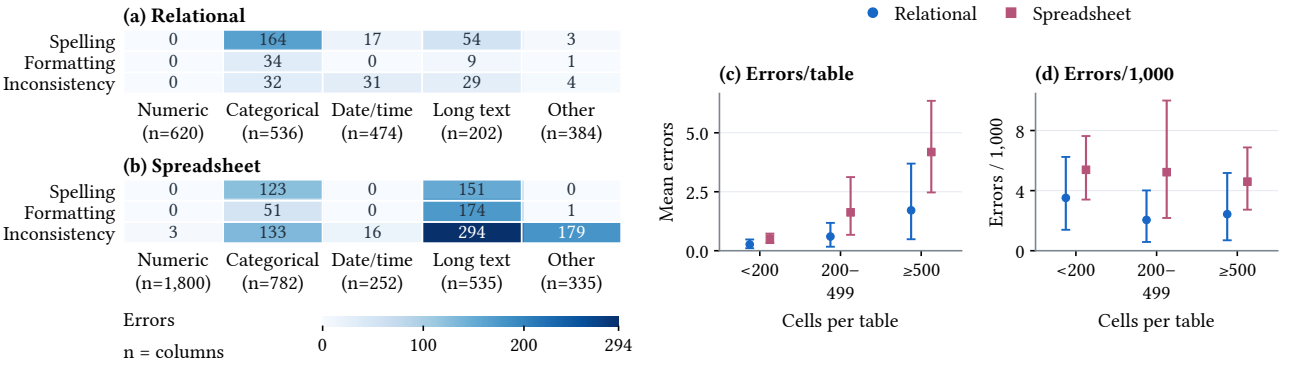


Figure 10: Error distributions by (a–b) column type, (c) mean errors per table by table size, and (d) mean error density by table size.

sources. While relational tables are generally cleaner with fewer errors, their most common error type is spelling, which may not be entirely surprising, as an inspection reveals that many of the tables with spelling errors are likely still manually entered by humans at some point upstream. Spreadsheet tables have substantially more inconsistency errors (6x) and formatting errors (5x) than relational tables, reflecting their more dynamic nature for ad-hoc editing. Spelling errors in spreadsheet tables are also more numerous in absolute terms.

We note that because one cell can have more than one error (e.g., a spelling error and a formatting one in the same cell), and therefore be assigned more than one error type, the sum of the error-type counts can exceed the total number of error cells.

Error distributions vary significantly by column type. To understand error distributions by column types, we infer each column's type from its header and values, into `numeric / date-time / long-text / categorical` (where `categorical` is for short and repeated text). We also have a fifth group, `Other`, for identifiers/codes and other mixed or unclassified columns.

Figure 10(a) and (b) show distributions of errors by these column types, in relational and spreadsheet tables, respectively. While `numeric` and `date-time` columns are generally clean with few errors, both `categorical` and `text` columns see many more errors across all three error-types. Columns in the `other` type are more likely to see inconsistency errors (e.g., ID pattern inconsistency for values in the same column).

Error incidence rates are similar across table sizes. Figure 10(c) shows the number of errors per table across tables of different sizes, and Figure 10(d) shows the same error incidence rate per 1000 cells (or error density) also across table sizes. While we can naturally expect more errors in larger tables as shown in Figure 10(c), their error incidence rate (or errors per cell) stays comparable across small and large tables, as shown in Figure 10(d), which suggests that large or small tables deserve equal attention when it comes to data cleaning.

Errors are unevenly distributed across table positions. To compare tables of different dimensions, we map the center of a cell with zero-based row and column indices (r, c) to relative coordinates $((c + 1/2)/C, (r + 1/2)/R)$, where R and C are the table's row and column counts. We use the 533 tables with at least five rows and five columns. Dividing each axis into five equal intervals produces the 5×5 regions in Figure 11. We normalize by the number of cells in each region because discrete table dimensions, especially narrow tables, give the regions different cell counts. Both counts exclude cells labeled ambiguous and cells explicitly skipped or excluded during human review.

Within this subset, the rightmost 20% of table width contains 35.5% of errors and 19.7% of eligible cells. Its error incidence is 7.37 per 1,000 cells, compared with 3.41 in the leftmost 20%. The same pattern holds within both sources: 5.94 versus 2.15 in relational tables, and 8.06 versus 4.02 in spreadsheets. Rates across the five row bands range from 3.51 to 4.34 per 1,000 cells, without a monotonic increase toward the bottom. Thus, errors are more frequent near the right edge of these tables, although column semantics and table composition may contribute to this association.

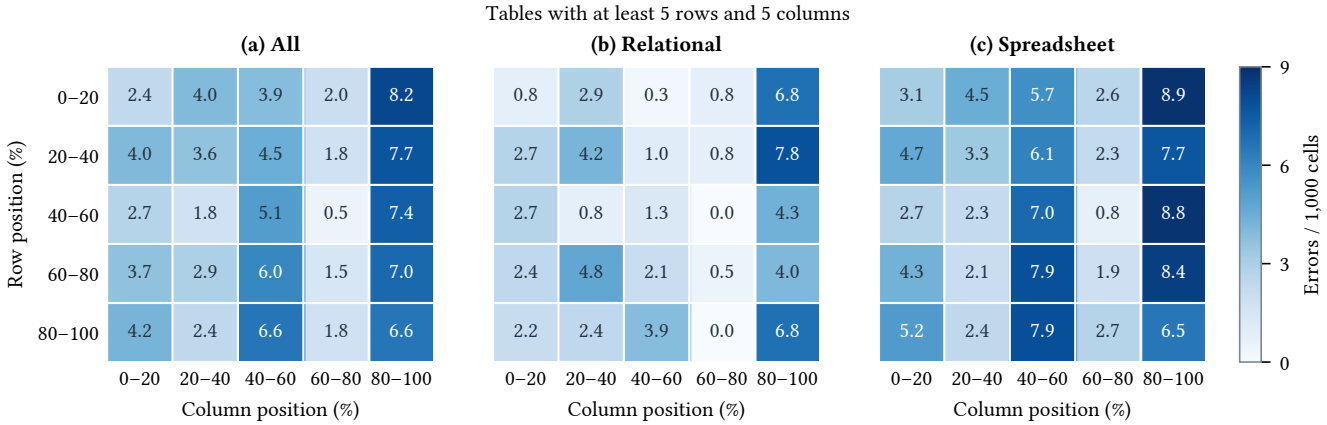


Figure 11: Error rates across table positions in the 533 tables with at least five rows and five columns. Darker colors indicate more errors per 1,000 cells.

5 CLEANBENCH: EXPERIMENTAL EVALUATION

We perform an extensive evaluation of existing data cleaning methods using CleanBench, to assess their ability to detect and repair real errors in the wild.

5.1 Experimental Setup

Tasks evaluated. Using CleanBench, we can evaluate data cleaning methods on 3 tasks: (1) detection-only, or how well a method can detect real errors; (2) repair-only, or how well a method performs repairs once errors are detected; and (3) end-to-end correction, which requires a method to both detect and repair correctly.

We report end-to-end correction in the main text and diagnostic results for detection only and repair only in Appendix C.

Evaluation metrics. We report prediction quality at the cell level[7]. A predicted repair is a true positive (TP) if it matches an accepted ground-truth repair at this cell location, with all other predictions counted as false positives (FP). An erroneous cell not repaired correctly counts as a false negative (FN). We compute Precision (P), Recall (R), and F1 respectively, as:[8]

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}.$$

We also report the false positive rate (FPR), calculated as:

$$\text{FPR} = \frac{FP_{\text{clean}}}{N_{\text{clean}}},$$

where FP_{clean} denotes the number of clean cells modified by an algorithm, and N_{clean} is the total number of clean cells under test. Intuitively, this calculates the fraction of clean cells incorrectly predicted as errors by an algorithm, which is important as most table cells (>99%) are clean.

We also report latency using wall-clock time, and monetary cost based on public API (for LLMs) and VM pricing (for CPU time).

[7]Table-level macro precision, recall, and F_1 for end-to-end correction are reported in Appendix D.
[8]Ambiguous cells are not counted and not used in the final metric calculation.

Methods evaluated. We evaluate an extensive collection of over 20 data cleaning methods that span different classes of methods.

Constraint-based data cleaning. We evaluate eight representative constraint-based methods, including BigDansing [38], Holistic [20], and NADEEF [22], etc., which use data quality constraints to detect violations and predict repairs.

Because real-world tables typically lack explicit constraints, we use a frontier LLM (GPT-5.5) to infer them from each table's semantics. We manually verify the inferred constraints on samples to ensure their validity before supplying them to all constraint-based methods.

Knowledge-based data cleaning. Methods like KATARA leverage external knowledge bases (e.g., YAGO and DBpedia) to identify errors and suggest repairs [21], which we also evaluate.

Spelling correction methods. Since spelling errors are a common type of error in tables, we evaluate two spelling correction methods, SymSpell, which is dictionary-based [27], and NeuSpell, which employs pretrained neural models [37].

LLM-prompt cleaning. We also compare two representative prompt-based cleaning methods: CleanAgent, which standardizes data values using LLM-generated code [56], and Cocoon, which uses LLMs to judge data validity and perform cleaning [70]. We keep their workflow structure, with the model substitutions and execution adaptations described in Appendix B.

LLM-agent self-optimized with feedback. Since LLM performance is highly sensitive to prompt wording, the results from a single hand-written prompt are unlikely to be representative of the model's true capabilities. We therefore report LLM performance using a multi-agent system with LLM self-optimized prompts.

Specifically, using a dev-set of 200 tables, we design a 3-agent system that can iteratively self-improve: (1) a prompt-writing agent that improves its prompt based on dev-set feedback; (2) a feedback agent that is instructed to perform error-analysis on the dev-set and give *generalized* feedback, with no specific examples so that the LLM cannot overfit by adding specific examples to

its prompt; and (3) a leakage-checking agent that sits in between the first two agents, to ensure that feedback given to the prompt-writing agent contains no example-specific leakage.

Starting from a simple initial prompt, we use the three-agent workflow with the fixed search budget described in Appendix B. We evaluate GPT-4.1, GPT-5.2, GPT-5.5, and GPT-6-Astra for three runs and report their average performance.

Coding agents. Given the versatile nature of coding agents, we also evaluate two coding agents, OpenAI Codex CLI [1] and GitHub Copilot CLI [2], using a frontier model GPT-6-Astra (results on earlier models such as GPT-4.1 and GPT-5.2 are not reported, as they are no longer available in these coding agents).

Commercial systems. Since data cleaning features are available in commercial spreadsheet systems, we also test Microsoft Excel's Clean Data [50] and Google Sheets' Smart Cleanup [32], using Playwright [5] to automatically extract predicted results.

Detailed configurations of evaluated methods appear in Appendix B.

5.2 End-to-End Task Results

Table 3 summarizes quality results of all methods on the end-to-end correction task. Detection-only and repair-only results appear in Appendix C. Table 3 includes available run-to-run variability for methods with repeated trials, including direct API models and coding agents.

Constraint and knowledge-based systems show limited effectiveness. While effective on some existing benchmarks, constraint-based and knowledge-based systems show limited effectiveness on CleanBench, with many false positives and limited recall. Our analysis shows that even when frontier LLMs are used to infer constraints (keys, functional dependencies, and denial constraints), the inferred constraints cover only a small fraction of columns and cells in real tables. [9]

Methods that use knowledge bases like YAGO face a similar limitation: most of the content in CleanBench is not represented in static knowledge bases, leading to limited or no coverage.

NLP spellers are less effective on tables. Spellers developed in the NLP literature that work well on natural language data are less effective on tables, which contain substantial domain-specific and proprietary content rarely found elsewhere (e.g., code names and abbreviations), leading to many false positives and low precision, which is consistent with prior studies that report similar results when using search engine spellers [65].

Commercial systems behave conservatively. Excel Clean Data achieves the highest precision among all methods because it proactively scans user data and surfaces only high-confidence suggestions, which makes it precision-oriented by design. Its coverage, however, is relatively limited.

Google Sheets Smart Cleanup produces no accepted repairs, as its current features primarily target specific operations, such as trimming leading/trailing whitespace, and adjusting cell formats.

Frontier LLMs are relatively more effective. LLM-based methods are more effective on CleanBench, as they possess strong world knowledge and reasoning capabilities. In the "LLM-agent optimized with feedback" category, we can see that result quality improves with each LLM generation, with GPT-6-Astra attaining the highest overall F1. Coding agents with GPT-6-Astra achieve similar recall but have lower precision scores.

[9] Real-world data often do not satisfy idealized constraints, e.g., canonical ones like `zip-code` → `city` are not observed in practice [6].

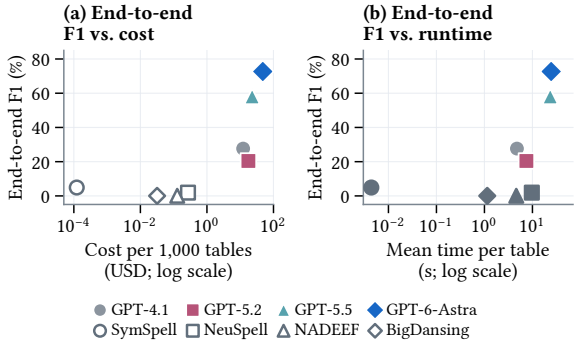


Figure 12: End-to-end F_1 versus cost (a) and runtime (b).

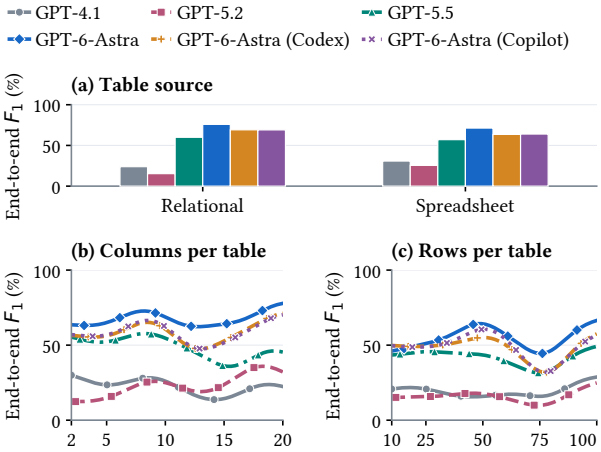


Figure 13: End-to-end correction across table characteristics

LLMs are significantly more expensive. Figure 12 compares end-to-end F_1 against estimated monetary cost[10] and runtime.

Figure 12(a) shows that LLM-based methods are significantly more expensive to run. GPT-6-Astra, the best-performing model, costs $47.70 per 1,000 tables, about 180 times the largest CPU cost estimate among these baselines (the x-axis is on a log scale). Figure 12(b) shows that GPT-6-Astra's runtime is 24.5 seconds per table, also significantly higher than other methods, showing a clear quality/cost tradeoff between these methods.

5.3 Sensitivity Analysis

We perform sensitivity analysis of result quality and cost, focusing on LLM-based methods given their strong results.

Difficulty stays comparable across table source or size. Figure 13(a) shows that relational and spreadsheet tables have comparable difficulty across all LLMs tested, while Figure 13(b) and (c)

[10] We estimate the cost of each method based on resource consumption. For LLMs, we estimate cost from inference usage and public API pricing [4], and runtime from per-table execution times. For CPU-based algorithms, we estimate computational costs based on CPU time at a rate of $0.10 per worker-hour [9].

Table 3: End-to-end correction results (%). Overall P/R/F_1 and FPR, with recall by error type. For methods with repeated trials, subscripts give ± sample standard deviation over the n completed independent runs shown beside each method; a dash means only one run is available for a direct API model or coding agent. Other single-run methods keep their point estimates. Lower FPR is better.

P/R/F_1 (%) 0 50 100

Method	Overall (%) P ↑	R ↑	F_1 ↑	FPR ↓	Spelling R ↑	Formatting R ↑	Inconsistency R ↑
Constraint-based data cleaning							
BigDansing (n=3)	**0.02**$_{\pm 0.00}$	**0.09**$_{\pm 0.00}$	**0.03**$_{\pm 0.00}$	**1.539**$_{\pm 0.000}$	0.0$_{\pm 0.0}$	0.0$_{\pm 0.0}$	0.2$_{\pm 0.0}$
Holistic (n=3)	**0.02**$_{\pm 0.00}$	**0.09**$_{\pm 0.00}$	**0.03**$_{\pm 0.00}$	**1.556**$_{\pm 0.000}$	0.0$_{\pm 0.0}$	0.0$_{\pm 0.0}$	0.2$_{\pm 0.0}$
DaisyP (n=3)	**0.00**$_{\pm 0.00}$	**0.00**$_{\pm 0.00}$	**0.00**$_{\pm 0.00}$	**0.004**$_{\pm 0.000}$	0.0$_{\pm 0.0}$	0.0$_{\pm 0.0}$	0.0$_{\pm 0.0}$
Horizon (n=3)	**0.01**$_{\pm 0.00}$	**0.09**$_{\pm 0.00}$	**0.02**$_{\pm 0.00}$	**2.736**$_{\pm 0.000}$	0.0$_{\pm 0.0}$	0.0$_{\pm 0.0}$	0.2$_{\pm 0.0}$
Relative Trust (n=3)	**0.00**$_{\pm 0.00}$	**0.00**$_{\pm 0.00}$	**0.00**$_{\pm 0.00}$	**1.708**$_{\pm 0.002}$	0.0$_{\pm 0.0}$	0.0$_{\pm 0.0}$	0.0$_{\pm 0.0}$
NADEEF (n=3)	**0.08**$_{\pm 0.00}$	**0.09**$_{\pm 0.00}$	**0.09**$_{\pm 0.00}$	**0.341**$_{\pm 0.000}$	0.0$_{\pm 0.0}$	0.0$_{\pm 0.0}$	0.2$_{\pm 0.0}$
LLUNATIC (n=3)	**0.05**$_{\pm 0.00}$	**0.18**$_{\pm 0.00}$	**0.08**$_{\pm 0.00}$	**1.122**$_{\pm 0.000}$	0.0$_{\pm 0.0}$	0.0$_{\pm 0.0}$	0.5$_{\pm 0.0}$
UniClean (n=3)	**0.06**$_{\pm 0.00}$	**0.09**$_{\pm 0.00}$	**0.07**$_{\pm 0.00}$	**0.472**$_{\pm 0.000}$	0.0$_{\pm 0.0}$	0.0$_{\pm 0.0}$	0.2$_{\pm 0.0}$
Knowledge-based data cleaning							
KATARA (KB) (n=3)	**0.00**$_{\pm 0.00}$	**0.00**$_{\pm 0.00}$	**0.00**$_{\pm 0.00}$	**3.110**$_{\pm 0.000}$	0.0$_{\pm 0.0}$	0.0$_{\pm 0.0}$	0.0$_{\pm 0.0}$
Spelling correction methods							
SymSpell (n=3)	**2.65**$_{\pm 0.00}$	**31.47**$_{\pm 0.00}$	**4.90**$_{\pm 0.00}$	**3.441**$_{\pm 0.000}$	67.2$_{\pm 0.0}$	0.0$_{\pm 0.0}$	3.7$_{\pm 0.0}$
NeuSpell (n=3)	**0.99**$_{\pm 0.00}$	**30.74**$_{\pm 0.00}$	**1.91**$_{\pm 0.00}$	**9.168**$_{\pm 0.000}$	56.2$_{\pm 0.0}$	17.0$_{\pm 0.0}$	12.3$_{\pm 0.0}$
LLM-prompt cleaning							
CleanAgent (n=3)	**0.15**$_{\pm 0.00}$	**4.67**$_{\pm 0.00}$	**0.29**$_{\pm 0.00}$	**9.299**$_{\pm 0.108}$	0.0$_{\pm 0.0}$	0.4$_{\pm 0.0}$	12.3$_{\pm 0.0}$
Cocoon (n=3)	**5.91**$_{\pm 1.08}$	**36.78**$_{\pm 7.63}$	**10.19**$_{\pm 1.90}$	**1.728**$_{\pm 0.042}$	56.6$_{\pm 11.4}$	18.1$_{\pm 5.2}$	27.6$_{\pm 6.7}$
LLM-agent optimized with feedback							
GPT-4.1 (n=3)	**22.37**$_{\pm 1.83}$	**36.41**$_{\pm 4.20}$	**27.65**$_{\pm 2.14}$	**0.377**$_{\pm 0.053}$	60.7$_{\pm 10.7}$	10.9$_{\pm 1.4}$	19.8$_{\pm 3.5}$
GPT-5.2 (n=3)	**12.48**$_{\pm 0.68}$	**56.21**$_{\pm 3.16}$	**20.41**$_{\pm 0.87}$	**1.184**$_{\pm 0.109}$	77.1$_{\pm 2.6}$	34.8$_{\pm 3.6}$	44.5$_{\pm 9.7}$
GPT-5.5 (n=3)	**55.90**$_{\pm 4.04}$	**60.72**$_{\pm 4.81}$	**58.15**$_{\pm 3.71}$	**0.133**$_{\pm 0.021}$	81.3$_{\pm 0.1}$	46.2$_{\pm 4.3}$	45.0$_{\pm 12.6}$
GPT-6-Astra (n=3)	**72.27**$_{\pm 2.67}$	**73.16**$_{\pm 0.89}$	**72.69**$_{\pm 1.16}$	**0.070**$_{\pm 0.012}$	85.2$_{\pm 0.4}$	79.0$_{\pm 1.1}$	59.2$_{\pm 2.1}$
Coding agents							
GPT-6-Astra (Codex) (n=3)	**57.38**$_{\pm 0.83}$	**76.03**$_{\pm 0.87}$	**65.40**$_{\pm 0.82}$	**0.156**$_{\pm 0.004}$	84.8$_{\pm 0.5}$	80.9$_{\pm 2.8}$	66.3$_{\pm 0.1}$
GPT-6-Astra (Copilot) (n=3)	**58.70**$_{\pm 0.97}$	**76.30**$_{\pm 0.48}$	**66.35**$_{\pm 0.74}$	**0.147**$_{\pm 0.006}$	85.2$_{\pm 0.1}$	80.7$_{\pm 2.9}$	66.5$_{\pm 0.6}$
Commercial systems							
Microsoft Excel Clean Data	**76.16**	**25.43**	**38.13**	**0.0221**	36.9	7.8	24.9
Google Sheets Smart Cleanup	**0.00**	**0.00**	**0.00**	**3.0373**	0.0	0.0	0.0

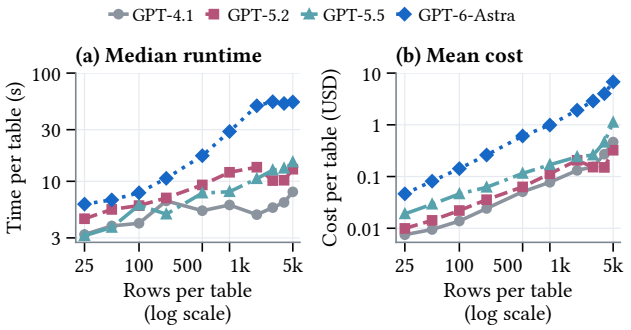


Figure 14: Runtime and cost as input table size grows.

show no systematic decline in quality as the number of columns and rows increases.

These results suggest that neither table source nor table dimensions consistently predict task difficulty.

Cost and runtime grow quickly on larger tables. We vary input tables from 25 to 5,000 rows and inspect the resulting cost/runtime numbers. Figure 14(a) shows that while runtime generally increases with table size, its growth becomes less pronounced at larger input sizes, potentially reflecting amortized latency and prompt caching.

Figure 14(b), on the other hand, shows a substantially greater increase in inference cost. GPT-6-Astra's mean cost rises from $0.046 to $6.77 per table. Unlike runtime, which grows more slowly and levels off for larger inputs, inference cost continues to increase substantially, likely because reasoning on more complex tables can incur additional token costs and require additional tool calls.

5.4 Error Analysis

We conduct an error analysis to understand where current methods still fall short, as even the strongest model, GPT-6-Astra, reaches only about 70% accuracy, which is far from perfect.

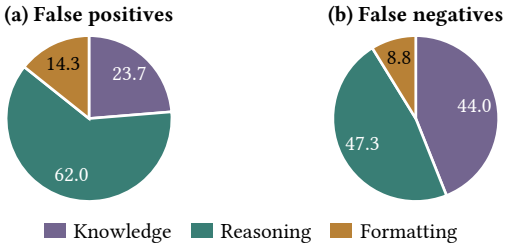


Figure 15: Error categorization of GPT-6-Astra, for false positive and false negative errors

Table 4: Mean end-to-end outcomes per run at labelled and clean cells, using the same runs as Table 3.

Method	Correct-repairs	Edits-on-clean	Unchanged-errors	Wrong-repairs
BigDansing	1	5,579	1,075	17
NADEEF	1	1,235	1,090	2
GPT-6-Astra	799.7	255	240.3	53
GPT-6-Astra (Codex CLI)	831	565	209.7	52.3
GPT-6-Astra (Copilot CLI)	834	534.3	206.3	52.7

Table 4 compares selected methods by their correct repairs (TP), edits on clean cells (FP), unchanged errors (FN), as well as wrong repairs on erroneous cells (count as both FP and FN).[11]

GPT-6-Astra with agent feedback makes an average of 255 edits on clean cells, leaving 240.3 erroneous cells unchanged and making 53 incorrect edits. Unchanged errors therefore account for 81.9% of its false negatives, indicating that missed corrections arise primarily from leaving error values untouched rather than proposing incorrect edits. Coding agents using GPT-6-Astra (Codex and Copilot) behave similarly, though both have many more clean edits than GPT-6-Astra with agent feedback, showing the need to calibrate predictions for data cleaning based on feedback, even for powerful models like GPT-6-Astra. In comparison, constraint-based methods BigDansing and NADEEF correctly repair few errors, and edit many clean cells due to the rigid nature of logic-based constraints.

Failure categorization. Following [45], we classify an incorrect prediction (false positive or false negative) into three main categories based on the information needed to make correct predictions:

- (1) *knowledge*, which requires a method to possess world knowledge in order to perform correct cleaning. Figure 16 (a) shows such an example, where a misspelled book title uses "`Pesto`", which should be "`Presto`" in this book title. Because both "`Pesto`" and "`Presto`" are valid words, this error can be corrected only if a method has the knowledge of the correct book name.

- (2) *reasoning*, which requires a method to reason across rows and columns in the table to make a correct determination. Figure 5(c) shows one such example: a method needs to reason about the implicit relationship between the "`Payee`" and "`Payee ID`" columns, to determine that cell "`97B`" is incorrect and should be "`95416`". Similarly, Figure 5(e) shows a case that requires comparing two rows from separate tables, to infer that one "`last-name`" is incorrect and should be "`Lederman`".

- (3) *formatting*, which concerns surface-level issues such as punctuation and whitespace. Figure 16(b) shows an example, where in an ocean of unfamiliar content, Astra leaves a trailing comma in the vendor name "`MATTHEW R. DEKOATZ,`" untouched, which is a missed error.

[11]Note that because we permit bi-directional repairs, e.g., "`Gcc`" and "`GCC`" in both directions in Figure 5(b), in our benchmark labels both "`Gcc`" and "`GCC`" are counted as error cells, though in this error-analysis of predictions from an algorithm, as long as one side is corrected we would not require repairs from the other side, leading to a lower total number than the 1,409 labeled errors reported earlier.

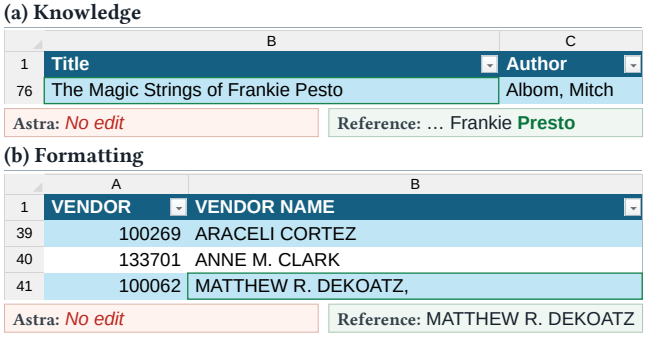


Figure 16: Example Errors for knowledge and formatting

Figure 15 shows an LLM-predicted categorization of errors for GPT-6-Astra, for both false positives and false negatives.

World knowledge plays a big role in false-negatives. Figure 15 shows that knowledge accounts for 44.0% of false negatives. This corresponds to examples in Figure 16, that are only fixable if a method or model has the knowledge of the correct book title. Even a powerful model like GPT-6-Astra can miss cases like this frequently (with other methods performing even worse), which is a clear gap in terms of prediction quality.

Reasoning is important for both false positives and false negatives. Reasoning accounts for 62.0% of false positives and 47.3% of false negatives, both of which are significant. Cases like Figure 5(c) and Figure 5(e) require holistic reasoning across broad table contexts (cross-column, rows and tables), which can often be missed in large and complex tables, leading to many false positives and false negatives.

Inconsistency across runs also contributes to failures. In the case of Figure 16(b), Astra correctly picks up the issue in the first run, but then fails to detect the same issue in subsequent runs. Similar to humans, in complex and large tables, minor details can often be lost, causing false-negatives. Improving the reliability of LLM-based data-cleaning systems (without expensive and repeated re-runs) is another area for future improvement.

6 CONCLUSION

We present the first large-scale benchmark with real tables and real errors, CleanBench, exhaustively labeled with quality guarantees. Our evaluation using CleanBench reveals where existing approaches each fall short: e.g., constraint-based algorithms miss errors outside their inferred relationships, while LLMs reach

higher accuracy at much greater inference cost. We release our benchmark as well as our labeling system for future research.

REFERENCES

[1] [n.d.]. Codex CLI. https://learn.chatgpt.com/docs/codex/cli.

[2] [n.d.]. Copilot CLI. https://github.com/github/copilot-cli.

[3] [n.d.]. Full technical report version of our paper, as well as code and data. https://github.com/kckevinchen/CleanBench.

[4] [n.d.]. OpenAI pricing. https://developers.openai.com/api/docs/pricing. Accessed: October 2026.

[5] [n.d.]. Playwright automation. https://playwright.dev/.

[6] [n.d.]. USPS City Versus Zip. https://www.census.gov/programs-surveys/geography/guidance/geo-areas/usps_census_city.html?utm_source=chatgpt.com.

[7] Ziawasch Abedjan, Xu Chu, Dong Deng, Raul Castro Fernandez, Ihab F Ilyas, Mourad Ouzzani, Paolo Papotti, Michael Stonebraker, and Nan Tang. 2016. Detecting data errors: where are we and what needs to be done? *Proceedings of the VLDB Endowment* 9, 12 (2016), 993–1004.

[8] Fatemeh Ahmadi, Marc Speckmann, Malte Fabian Kuhlmann, and Ziawasch Abedjan. 2025. MaTElDa: Multi-Table Error Detection. In *Proceedings of the 28th International Conference on Extending Database Technology*. https://www.openproceedings.org/2025/conf/edbt/paper-98.pdf

[9] Amazon Web Services. [n.d.]. Amazon EC2 T3 Instances. https://aws.amazon.com/ec2/instance-types/t3/. Accessed: October 2026.

[10] Patricia C. Arocena, Boris Glavic, Giansalvatore Mecca, Renée J. Miller, Paolo Papotti, and Donatello Santoro. 2015. Messing Up with BART: Error Generation for Evaluating Data-Cleaning Algorithms. *Proceedings of the VLDB Endowment* 9, 2 (2015), 36–47. https://doi.org/10.14778/2850578.2850579

[11] George Beskales, Ihab F. Ilyas, Lukasz Golab, and Artur Galiullin. 2013. On the Relative Trust between Inconsistent Data and Inaccurate Constraints. In *2013 IEEE 29th International Conference on Data Engineering*. 541–552. https://cs.uwaterloo.ca/~ilyas/papers/BeskalesICDE2013.pdf

[12] Divya Bhadauria, Hazar Harmouch, Felix Naumann, Divesh Srivastava, and Lisa Ehrlinger. 2026. A Catalog of Data Errors. *arXiv preprint arXiv:2604.09277* (2026).

[13] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[14] John Bunge and Michael Fitzpatrick. 1993. Estimating the number of species: a review. *Journal of the American statistical Association* 88, 421 (1993), 364–373.

[15] Anne Chao. 1984. Nonparametric estimation of the number of classes in a population. *Scandinavian Journal of Statistics* 11, 4 (1984), 265–270.

[16] Anne Chao. 1987. Estimating the population size for capture-recapture data with unequal catchability. *Biometrics* 43, 4 (1987), 783–791.

[17] Qixu Chen, Yeye He, Raymond Chi-Wing Wong, Weiwei Cui, Song Ge, Haidong Zhang, Dongmei Zhang, and Surajit Chaudhuri. 2025. Auto-Test: Learning Semantic-Domain Constraints for Unsupervised Error Detection in Tables. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–27.

[18] Fei Chiang and Renée J. Miller. 2011. A Unified Model for Data and Constraint Repair. In *2011 IEEE 27th International Conference on Data Engineering*. 446–457. https://doi.org/10.1109/ICDE.2011.5767833

[19] Xu Chu, Ihab F Ilyas, Sanjay Krishnan, and Jiannan Wang. 2016. Data cleaning: Overview and emerging challenges. In *Proceedings of the 2016 international conference on management of data*. 2201–2206.

[20] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Holistic Data Cleaning: Putting Violations into Context. In *2013 IEEE 29th International Conference on Data Engineering*. 458–469. https://doi.org/10.1109/ICDE.2013.6544847

[21] Xu Chu, John Morcos, Ihab F. Ilyas, Mourad Ouzzani, Paolo Papotti, Nan Tang, and Yin Ye. 2015. KATARA: A Data Cleaning System Powered by Knowledge Bases and Crowdsourcing. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1247–1261. https://doi.org/10.1145/2723372.2749431

[22] Michele Dallachiesa, Amr Ebaid, Ahmed Eldawy, Ahmed Elmagarmid, Ihab F Ilyas, Mourad Ouzzani, and Nan Tang. 2013. NADEEF: a commodity data cleaning system. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. 541–552.

[23] Xiaoou Ding, Zekai Qian, Hongzhi Wang, Siying Chen, Yafeng Tang, Hongbin Su, Huan Hu, and Chen Wang. 2025. UniClean: A Scalable Data Cleaning Solution for Mixed Errors based on Unified Cleaners and Optimized Cleaning Workflow. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4117–4130. https://doi.org/10.14778/3749646.3749681

[24] Simao Eduardo, Alfredo Nazabal, Christopher K. I. Williams, and Charles Sutton. 2020. Robust Variational Autoencoders for Outlier Detection and Repair of Mixed-Type Data. In *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research)*, Vol. 108. 4056–4066. https://proceedings.mlr.press/v108/eduardo20a.html

[25] European Spreadsheet Risks Interest Group. [n.d.]. Spreadsheet mistakes – news stories. https://eusprig.org/research-info/horror-stories/.

[26] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional Functional Dependencies for Capturing Data Inconsistencies. *ACM Transactions on Database Systems* 33, 2, Article 6 (2008), 48 pages. https://doi.org/10.1145/1366102.1366103

[27] Wolf Garbe. [n.d.]. SymSpell: Symmetric Delete Spelling Correction. https://github.com/wolfgarbe/SymSpell. Reference software; accessed September 2026.

[28] Congcong Ge, Yunjun Gao, Xiaoye Miao, Bin Yao, and Haobo Wang. 2022. A Hybrid Data Cleaning Framework Using Markov Logic Networks. *IEEE Transactions on Knowledge and Data Engineering* 34, 5 (2022), 2048–2062. https://doi.org/10.1109/TKDE.2020.3012472

[29] Floris Geerts, Giansalvatore Mecca, Paolo Papotti, and Donatello Santoro. 2013. The LLUNATIC Data-Cleaning Framework. *Proceedings of the VLDB Endowment* 6, 9 (2013), 625–636. https://www.vldb.org/pvldb/vol6/p625-mecca.pdf

[30] Stella Giannakopoulou, Manos Karpathiotakis, and Anastasia Ailamaki. 2020. Cleaning Denial Constraint Violations through Relaxation. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 805–815. https://doi.org/10.1145/3318464.3389775

[31] Irving J Good. 1953. The population frequencies of species and the estimation of population parameters. *Biometrika* 40, 3-4 (1953), 237–264.

[32] Google. 2020. Google Sheets: Smart Cleanup Feature. https://workspace.google.com/blog/product-announcements/connected-sheets-is-generally-available. Accessed 2026-07-17.

[33] Ziyan Han, Jing Zhu, Jinbin Huang, Sifan Huang, Yaoshu Wang, Rui Mao, and Jianbin Qin. 2026. BClean+: A Bayesian Data Cleaning System With Automated Prior Generation. *IEEE Transactions on Knowledge and Data Engineering* 38, 4 (2026), 2165–2182. https://doi.org/10.1109/TKDE.2026.3656941

[34] Alireza Heidari, Joshua McGrath, Ihab F Ilyas, and Theodoros Rekatsinas. 2019. HoloDetect: few-shot learning for error detection. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*. 829–846. https://doi.org/10.1145/3299869.3319888

[35] Zhipeng Huang and Yeye He. 2018. Auto-detect: Data-driven error detection in tables. In *Proceedings of the 2018 International Conference on Management of Data*. 1377–1392.

[36] Ihab F. Ilyas and Xu Chu. 2019. *Data Cleaning*. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3310205

[37] Sai Muralidhar Jayanthi, Danish Pruthi, and Graham Neubig. 2020. NeuSpell: A Neural Spelling Correction Toolkit. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 158–164. https://doi.org/10.18653/v1/2020.emnlp-demos.21

[38] Zuhair Khayyat, Ihab F. Ilyas, Alekh Jindal, Samuel Madden, Mourad Ouzzani, Paolo Papotti, Jorge-Arnulfo Quiané-Ruiz, Nan Tang, and Si Yin. 2015. BigDansing: A System for Big Data Cleansing. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 1215–1230. https://doi.org/10.1145/2723372.2747646

[39] Sanjay Krishnan, Michael J Franklin, Ken Goldberg, and Eugene Wu. 2017. Boostclean: Automated error detection and repair for machine learning. *arXiv preprint arXiv:1711.01299* (2017).

[40] Sanjay Krishnan, Jiannan Wang, Eugene Wu, Michael J Franklin, and Ken Goldberg. 2016. Activeclean: Interactive data cleaning for statistical modeling. *Proc. VLDB Endow.* 9, 12 (2016), 948–959.

[41] Peng Li, Yeye He, Dror Yashar, Weiwei Cui, Song Ge, Haidong Zhang, Danielle Rifinski Fainman, Dongmei Zhang, and Surajit Chaudhuri. 2024. Table-gpt: Table fine-tuned gpt for diverse table tasks. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.

[42] Peng Li, Xi Rao, Jennifer Blase, Yue Zhang, Xu Chu, and Ce Zhang. 2021. Cleanml: A study for evaluating the impact of data cleaning on ml classification tasks. In *2021 IEEE 37th international conference on data engineering (ICDE)*. IEEE, 13–24.

[43] Xian Li, Xin Luna Dong, Kenneth Lyons, Weiyi Meng, and Divesh Srivastava. 2012. Truth Finding on the Deep Web: Is the Problem Solved? *Proceedings of the VLDB Endowment* 6, 2 (2012), 97–108. https://www.vldb.org/pvldb/vol6/p97-li.pdf

[44] Xinyuan Liu, Jiahui Chen, Bocheng Hu, Yu Sun, Xinyang Chen, Shaoxu Song, and Yongxin Tong. 2025. Towards Practical Benchmarking of Data Cleaning Techniques: On Generating Authentic Errors via Large Language Models. https://doi.org/10.48550/arXiv.2507.10934 arXiv:2507.10934 [cs.DB]

[45] Yurong Liu, Yeye He, Haoyu Dong, Junjie Xing, Shi Han, Dongmei Zhang, and Surajit Chaudhuri. 2026. Auto-Fill: Learning to Predict Missing Values Accurately with Specialist Language Models. *arXiv preprint arXiv:2607.19847* (2026).

[46] Mohammad Mahdavi and Ziawasch Abedjan. 2019. REDS: Estimating the Performance of Error Detection Strategies Based on Dirtiness Profiles. In *Proceedings of the 31st International Conference on Scientific and Statistical Database Management*. https://doi.org/10.1145/3335783.3335808

[47] Mohammad Mahdavi and Ziawasch Abedjan. 2020. Baran: Effective error correction via a unified context representation and transfer learning. *Proceedings of the VLDB Endowment* 13, 12 (2020), 1948–1961.

[48] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: a configuration-free error detection system. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*. 865–882. https://doi.org/10.1145/3299869.3324956

[49] Giansalvatore Mecca, Paolo Papotti, Donatello Santoro, and Enzo Veltri. 2024. BUNNI: Learning Repair Actions in Rule-driven Data Cleaning. *Journal of Data and Information Quality* 16, 2, Article 12 (2024), 31 pages.

[50] Microsoft. [n.d.]. Clean Data in Excel with Copilot. https://support.microsoft.com/en-us/office/clean-data-in-excel-7fe20d89-3f57-46d3-b659-e8f3ee853bda?ns=XLWAENDUSER&version=16. Accessed 2026-07-15.

[51] Felix Neutatz, Mohammad Mahdavi, and Ziawasch Abedjan. 2019. ED2: a case for active learning in error detection. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*. 2249–2252. https://doi.org/10.1145/3357384.3358129

[52] Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, and Jianwei Yin. 2024. Automatic Data Repair: Are We Ready to Deploy? *Proceedings of the VLDB Endowment* 17, 10 (2024), 2617–2630.

[53] Wei Ni, Kaihang Zhang, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Yaoshu Wang, and Jianwei Yin. 2025. ZeroED: Hybrid Zero-shot Error Detection through Large Language Model Reasoning. *arXiv preprint arXiv:2504.05345* (2025). https://doi.org/10.48550/arXiv.2504.05345

[54] Raymond R. Panko. 2008. Spreadsheet Errors: What We Know. What We Think We Can Do. *arXiv preprint arXiv:0802.3457* (2008). https://doi.org/10.48550/arXiv.0802.3457

[55] Jinfeng Peng, Derong Shen, Nan Tang, Tieying Liu, Yue Kou, Tiezheng Nie, Hang Cui, and Ge Yu. 2022. Self-supervised and Interpretable Data Cleaning with Sequence Generative Adversarial Networks. *Proceedings of the VLDB Endowment* 16, 3 (2022), 433–446. https://doi.org/10.14778/3570690.3570694

[56] Danrui Qi, Zhengjie Miao, and Jiannan Wang. 2024. CleanAgent: Automating Data Standardization with LLM-based Agents. *arXiv preprint arXiv:2403.08291* (2024). https://doi.org/10.48550/arXiv.2403.08291

[57] Erhard Rahm and Hong Hai Do. 2000. Data cleaning: Problems and current approaches. *IEEE Data Eng. Bull.* 23, 4 (2000), 3–13.

[58] Joeri Rammelaere and Floris Geerts. 2018. Explaining Repaired Data with CFDs. *Proceedings of the VLDB Endowment* 11, 11 (2018), 1387–1399. https://doi.org/10.14778/3236187.3236193

[59] Theodoros Rekatsinas, Xu Chu, Ihab F Ilyas, and Christopher Ré. 2017. HoloClean: holistic data repairs with probabilistic inference. *Proceedings of the VLDB Endowment* 10, 11 (2017), 1190–1201.

[60] El Kindi Rezig, Mourad Ouzzani, Walid G. Aref, Ahmed K. Elmagarmid, Ahmed R. Mahmood, and Michael Stonebraker. 2021. Horizon: Scalable Dependency-driven Data Cleaning. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2546–2554. https://doi.org/10.14778/3476249.3476301

[61] Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, and Andreas Grafberger. 2018. Automating Large-Scale Data Quality Verification. *Proceedings of the VLDB Endowment* 11, 12 (2018), 1781–1794. https://doi.org/10.14778/3229863.3229867

[62] Jie Song and Yeye He. 2021. Auto-validate: Unsupervised data validation using data-domain patterns inferred from data lakes. In *Proceedings of the 2021 International Conference on Management of Data*. 1678–1691.

[63] Beth Trushkowsky, Tim Kraska, Michael J. Franklin, and Purnamrita Sarkar. 2013. Crowdsourced Enumeration Queries. In *2013 IEEE 29th International Conference on Data Engineering*. 673–684. https://doi.org/10.1109/ICDE.2013.6544865

[64] Dezhan Tu, Yeye He, Weiwei Cui, Song Ge, Haidong Zhang, Shi Han, Dongmei Zhang, and Surajit Chaudhuri. 2023. Auto-validate by-history: Auto-program data quality constraints to validate recurring data pipelines. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4991–5003.

[65] Pei Wang and Yeye He. 2019. Uni-Detect: a unified approach to automated error detection in tables. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*. 811–828. https://doi.org/10.1145/3299869.3319855

[66] Yibo Wang, Riteng Zhang, Yinghao He, Yongye Su, Bharat Bhargava, and Chunwei Liu. 2026. TabClean: Reusable LLM-Synthesized Programs for Tabular Data Cleaning. *arXiv preprint arXiv:2606.25388* (2026). https://doi.org/10.48550/arXiv.2606.25388

[67] Mohamed Yakout, Laure Berti-Équille, and Ahmed K Elmagarmid. 2013. Don't be scared: use scalable automatic repairing with maximal likelihood and bounded changes. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. 553–564.

[68] Mohamed Yakout, Ahmed K Elmagarmid, Jennifer Neville, Mourad Ouzzani, and Ihab F Ilyas. 2011. Guided data repair. *arXiv preprint arXiv:1103.3103* (2011).

[69] Chenyu Yang, Yuyu Luo, Chuanxuan Cui, Ju Fan, Chengliang Chai, and Nan Tang. 2025. Data imputation with limited data redundancy using data lakes. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3354–3367.

[70] Shuo Zhang, Zezhou Huang, and Eugene Wu. 2024. Data Cleaning Using Large Language Models. *arXiv preprint arXiv:2410.15547* (2024). https://doi.org/10.48550/arXiv.2410.15547

[71] Mark Ziemann, Yotam Eren, and Assam El-Osta. 2016. Gene name errors are widespread in the scientific literature. *Genome Biology* 17 (2016), 177. https://doi.org/10.1186/s13059-016-1044-7

[72] Justin Zobel. 1998. How Reliable Are the Results of Large-Scale Information Retrieval Experiments?. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 307–314. https://doi.org/10.1145/290941.291014

A CANDIDATE-GENERATION PROMPT EXAMPLES

Figures 17–21 illustrate the prompts used for spelling, formatting, and the three forms of inconsistency. The prompts are shortened for readability, with miniature inputs and illustrative outputs as examples. These outputs are used as candidates for human verification.

Like discussed in Section 3.3, the goal of these prompts is to exhaustively flag all candidates, which we empirically validate to catch almost all true-errors in an extensive manual audit in Section 3.5.

Spelling errors

Prompt:

```
Task: Find every cell that may contain a spelling error. Use the column header and values as context. Favor recall:
     include plausible misspellings even when uncertain, and indicate low confidence. Return JSON with the row, column,
     observed value, explanation, suggested correction, and confidence. Return {"issues": []} if no candidate is found.

Input table (row indices are zero-based):
| __row__ | Specialty   |
|---------|-------------|
| 0       | Acupunture  |
| 1       | Cardiology  |
| 2       | Oncology    |
```

Illustrative completion:

```
{"issues": [{"row": 0, "columns": ["Specialty"],
  "observed": "Acupunture", "explanation": "A letter is missing from 'Acupuncture'.",
  "corrected_value": {"Specialty": "Acupuncture"}, "confidence": "high"}]}
```

Figure 17: Example prompt for spelling candidate generation.

Formatting errors

Prompt:

```
Task: Find every cell with a possible punctuation or spacing defect. Examples include "St.. Louis" -> "St. Louis",
     "Sales,Marketing" -> "Sales, Marketing", and "James   Smith" -> "James Smith". These examples are not exhaustive.
     Flag other suspicious punctuation or spacing supported by a standard rule or a clear column convention. Return JSON
     with the row, column, observed value, explanation, suggested correction, and confidence.

Input table (row indices are zero-based):
| __row__ | Department           |
|---------|----------------------|
| 0       | Sales                |
| 1       | Research,Development |
| 2       | Operations           |
```

Illustrative completion:

```
{"issues": [{"row": 1, "columns": ["Department"],
  "observed": "Research,Development", "explanation": "The comma is missing a following space.",
  "corrected_value": {"Department": "Research, Development"}, "confidence": "high"}]}
```

Figure 18: Example prompt for formatting candidate generation.

Single-column inconsistency

Prompt:

```
Task: Inspect the distinct values and their frequencies in the Month column. Find groups that express the same value
     differently, or use a minority style that conflicts with the column's dominant style. Do not group unrelated values
     merely because they look similar. Return JSON listing each group's values and a canonical form when one is supported.
     Return {"clusters": []} if no inconsistency is found.

Input column:
| Value | Count |
|-------|-------|
| Jan   | 12    |
| Feb   | 10    |
| Mar   | 11    |
| Jul   | 9     |
| July  | 1     |
```

Illustrative completion:

```
{"clusters": [{"values": ["Jul", "July"], "kind": "text", "canonical": "Jul"}]}
```

The group links two representations of the same month; the prevailing abbreviated form suggests reviewing cells containing `July`.

Figure 19: Example prompt for single-column inconsistency.

Cross-column inconsistency

Prompt:

```
Task: Check for inconsistencies in how strings in different columns relate to one another. Infer plausible relationships
     from the headers and rows, and flag every possible violation for human review. Return JSON with the row, involved
     columns, observed values, explanation, suggested correction if supported, and confidence. Leave the correction null
     when the conflicting columns do not establish which value is wrong.

Input table (row indices are zero-based):
| __row__ | Payee              | Payee ID |
|---------|--------------------|----------|
| 0       | 1001 - East Office| 1001     |
| 1       | 1002 - West Office| 1002     |
| 2       | 95416 - North Team| 95146    |
```

Illustrative completion:

```
{"issues": [{"row": 2, "columns": ["Payee", "Payee ID"],
  "observed": "95416 - North Team / 95146",
  "explanation": "The prefix 95416 disagrees with Payee ID 95146.",
  "corrected_value": null, "confidence": "medium"}]}
```

Figure 20: Example prompt for cross-column string inconsistency.

Cross-table inconsistency

Prompt:

```
Task: Identify possible inconsistencies across the input tables. Infer which columns describe comparable attributes and
     which records refer to the same entity from the table names, headers, and values. Check linked records for
     conflicting attribute values. Favor recall: keep plausible conflicts and uncertain matches for human review. Return
     JSON with the table names and local row indices of the involved records, the conflicting columns, value-level
     evidence, rationale, verdict, and confidence. Do not assume which value is correct without supporting evidence.
     Return {"findings": []} if no candidate is found.

Input tables (row indices are local to each table and zero-based):
Table A:
| __row__ | Email            | First name | Last name |
|---------|------------------|------------|-----------|
| 0       | alex@example.org | Alex       | Lederman  |

Table B:
| __row__ | Email            | First name | Last name |
|---------|------------------|------------|-----------|
| 0       | alex@example.org | Alex       | Ledderman |
```

Illustrative completion:

```
{"findings": [{"kind": "attribute_conflict", "verdict": "uncertain",
  "locations": [{"sheet": "A", "row": 0}, {"sheet": "B", "row": 0}],
  "columns": ["Last name"], "evidence": "Lederman / Ledderman",
  "rationale": "Matching email and first name link records with different surnames.",
  "confidence": "medium"}]}
```

Figure 21: Example prompt for cross-table inconsistency with both source locations preserved.

B DETAILED METHOD CONFIGURATIONS

Implementation details and sources for non-LLM methods. For BigDansing, Holistic, DaisyP, Horizon, and Relative Trust, we use the re-implementations of these methods developed in a prior study [52]; For NADEEF we use its Java FD core, for LLUNATIC we use its native chase with automatic greedy conflict resolution, and for UniClean we use its local, no-sampling FD workflow. All eight constraint-based methods use table-specific constraints inferred by GPT-5.5, with each method's native engine selecting repairs. For BigDansing and Holistic, functional dependencies are translated into denial constraints. SymSpell uses its reference dictionary implementation and NeuSpell its pretrained checker, without fitting to benchmark labels. Their repair-only configurations run inference at the supplied locations. KATARA uses the released KB component and its public knowledge-base files. The additional detection-only methods use histogram outlier detection (dBoost), syntactic and statistical missing-value checks (FAHES), and released semantic-domain constraints (Auto-Test).

LLM prompting workflows. CleanAgent uses the annotation, programming, and execution agents with the `runtime_repair_v1` adaptation: it checks output validity, permits limited retries after execution errors, supplies documentation for the installed DataPrep API, and preserves string values and table coordinates. Its standardization types are email, phone, date, country, and IP address. Cocoon uses its connected unusual-value detection and string-standardization workflow; repair only uses its SQL-formatting variant with known locations. Both workflows use GPT-4o-2024-11-20, which are the closest we can find at the time of our experiments, to the original models used in these two methods, namely GPT-4o-2024-08-06 in CleanAgent and GPT-4 Turbo in Cocoon. The Cocoon configuration allows 90 seconds per request and 900 seconds per table. Detection is obtained from the recorded edits when a workflow returns only a cleaned table.

LLM-agent optimized with feedback. Each request receives one complete dirty table with headers and literal cell strings. GPT-5.2 and GPT-5.5 use reasoning effort `none`; GPT-6-Astra uses `low`. The GPT-5.5 prompt writer runs three rounds with three accepted candidates per round. Selection maximizes development detection F_1 among candidates with valid responses for all 200 development tables, keeping the earliest candidate on ties. The selected prompt is fixed before comparison across the four target models. Detection and end-to-end correction share the resulting responses.

Repair-only API requests instead receive eligible error coordinates without accepted replacements. Prompt selection is separate for each target model, using the 32 development tables with 210 replacement targets. Each candidate is evaluated three times; the selected prompt maximizes mean development F_1 among completely executed candidates, with the base prompt winning ties. The candidate pools contain seven generic strategies per model, except for GPT-5.2's expanded pool of 14. The final comparison covers all 1,093 targets and reports three repetitions.

Coding-agent settings. The Astra configurations use Codex CLI 0.158.0 and Copilot CLI 1.0.88, both with `low` reasoning effort and a 300-second limit per table. For each configuration, we run three independent end-to-end repetitions with one session at a time. Each isolated session receives a read-only dirty CSV and the error definitions, with Python, pandas, NumPy, and SciPy available. The task prohibits web search, package installation, calls to other models, and additional agents. Outputs must pass JSON, coordinate, and original-value validation. Detection and end-to-end correction share a session's findings; repair only uses separate sessions with coordinates but no reference answers.

Commercial interfaces and execution accounting. We use Excel's AutoClean (Clean Data) tool to detect errors and apply its suggested fixes. Playwright captures the suggested cell locations and replacement values, which are mapped back to the input table coordinates for evaluation. Google Sheets detection expands row- or column-level suggestions to cell coordinates; correction uses recorded Cleanup Apply values. Neither interface has a reported repair-only configuration. Capture hashes and input-grid checks bind these outputs to the frozen tables. Every selected repetition and failed table remains in its task's scoring population. Invalid or absent outputs provide no predictions; failures are not treated as evidence that a table is clean. Exact source revisions, environment manifests, method-specific time limits, and capture records are saved with the selected-run artifacts.

C DETECTION-ONLY AND REPAIR-ONLY RESULTS

Tables 5 and 6 report the results of our two sub-tasks, detection-only and repair-only, respectively, which are additional diagnostic tasks on top of our end-to-end evaluations.

Detection-only evaluates all 1,409 labeled error locations; a true positive requires identifying the location, irrespective of the proposed replacement. Repair-only supplies the error locations to a method and evaluates accepted replacements at the 1,093 cells that require a value change.

In general, we can see that Tables 5 shows similar results as Tables 3 across all methods, while Tables 6 shows much higher quality results, suggesting that detecting errors in an ocean of table content (without producing many false-positives) is likely the key challenge in the end-to-end task. Repair-only at a given cell location known to contain errors, is likely much easier, as the higher quality scores would suggest, since LLMs are already quite reliable in performing a similar task of predicting the value of a missing table cell, like studied in [45].

Table 5: Detection-only results (%). Overall P/R/F_1 and clean-cell FPR, with recall by error type, on 1,409 labeled error cells. Scores are averaged across evaluation runs. Lower FPR is better.

P/R/F_1 (%) 0 50 100

Method	Overall (%)				Spelling	Formatting	Inconsistency
	P ↑	R ↑	F_1 ↑	FPR ↓	R ↑	R ↑	R ↑
Statistical checks							
dBoost	**0.72**	**13.34**	**1.37**	**7.1216**	6.2	7.4	20.0
FAHES	**0.54**	**15.76**	**1.04**	**11.3545**	4.3	0.7	27.5
Constraint-based data cleaning							
Auto-Test	**7.02**	**1.49**	**2.46**	**0.0768**	2.9	0.0	0.8
BigDansing	**0.75**	**2.48**	**1.15**	**1.2840**	2.0	1.5	3.6
Holistic	**0.45**	**1.14**	**0.64**	**0.9807**	1.6	0.4	1.1
DaisyP	**0.46**	**9.37**	**0.87**	**7.9041**	4.1	7.4	14.6
Horizon	**0.12**	**0.14**	**0.13**	**0.4790**	0.0	0.0	0.3
Relative Trust	**0.00**	**0.00**	**0.00**	**0.9377**	0.0	0.0	0.0
NADEEF	**0.47**	**9.51**	**0.89**	**7.8942**	4.1	7.4	14.8
LLUNATIC	**0.29**	**0.85**	**0.44**	**1.1223**	0.0	0.4	1.5
UniClean	**0.64**	**0.78**	**0.70**	**0.4724**	0.2	0.0	1.5
Knowledge-based data cleaning							
KATARA (KB)	**0.15**	**14.12**	**0.29**	**37.0344**	15.0	4.1	17.1
Spelling correction methods							
SymSpell	**3.86**	**35.56**	**6.96**	**3.4474**	83.4	16.3	6.4
NeuSpell	**2.80**	**67.78**	**5.37**	**9.1602**	87.5	94.4	47.7
LLM-prompt cleaning							
CleanAgent	**0.34**	**8.07**	**0.65**	**9.2990**	0.0	0.6	15.5
Cocoon	**8.60**	**41.87**	**14.27**	**1.7284**	62.6	36.2	32.1
LLM-agent optimized with feedback							
GPT-4.1	**16.89**	**35.11**	**22.76**	**0.6720**	62.7	13.2	22.3
GPT-5.2	**8.51**	**54.84**	**14.73**	**2.2963**	81.1	37.3	43.7
GPT-5.5	**45.66**	**53.61**	**49.30**	**0.2487**	86.4	50.1	33.2
GPT-6-Astra	**70.87**	**65.65**	**68.15**	**0.1051**	88.8	87.4	45.1
Coding agents							
GPT-6-Astra (Codex)	**58.67**	**67.21**	**62.65**	**0.1841**	87.9	88.1	48.4
GPT-6-Astra (Copilot)	**58.67**	**68.20**	**63.08**	**0.1868**	88.7	88.1	49.8
Commercial systems							
Microsoft Excel Clean Data	**81.22**	**24.56**	**37.71**	**0.0221**	38.3	7.8	22.5
Google Sheets Smart Cleanup	**0.22**	**2.06**	**0.40**	**3.5748**	0.0	1.5	3.5

Table 6: Repair-only results (%), with error locations supplied. Overall P/R/F_1 and recall by error type cover 1,093 cells requiring a value change. Scores are averaged across evaluation runs. Only methods with a reported repair-only configuration are included.

P/R/F_1 (%) 0 50 100

Method	Overall (%)			Spelling	Formatting	Inconsistency
	P ↑	R ↑	F_1 ↑	R ↑	R ↑	R ↑
Constraint-based data cleaning						
BigDansing	**36.36**	**0.73**	**1.43**	0.2	0.0	2.0
Holistic	**30.00**	**0.55**	**1.08**	0.2	0.0	1.5
DaisyP	**0.00**	**0.00**	**0.00**	0.0	0.0	0.0
Horizon	**54.55**	**0.55**	**1.09**	0.0	0.0	1.5
Relative Trust	**0.00**	**0.00**	**0.00**	0.0	0.0	0.0
NADEEF	**80.00**	**0.73**	**1.45**	0.4	0.0	2.0
Spelling correction methods						
SymSpell	**70.35**	**31.47**	**43.49**	67.2	0.0	3.7
NeuSpell	**40.63**	**30.74**	**35.00**	56.2	17.0	12.3
LLM-prompt cleaning						
CleanAgent	**57.17**	**13.66**	**21.68**	14.3	6.7	16.0
Cocoon	**75.41**	**68.01**	**71.51**	85.2	45.3	60.3
LLM-agent optimized with feedback						
GPT-4.1	**89.00**	**84.84**	**86.87**	92.5	70.4	86.6
GPT-5.2	**92.34**	**88.20**	**90.22**	93.1	74.9	91.9
GPT-5.5	**92.49**	**90.94**	**91.71**	93.2	86.7	92.1
GPT-6-Astra	**94.23**	**93.72**	**93.98**	94.0	89.6	97.0
Coding agents						
GPT-6-Astra (Codex)	**93.66**	**91.86**	**92.75**	92.6	85.6	96.3
GPT-6-Astra (Copilot)	**93.83**	**91.86**	**92.83**	92.8	85.6	96.3

D TABLE-LEVEL END-TO-END RESULTS

Recall that our results in Table 3 gives each cell equal weight when calculating precision, recall, and F_1, which in ML is known as "micro-averaged" results. Since such micro results may give tables with many errors (or receive many false-positive predictions) higher weight, we also report "macro-averaged" results, where each table receives equal weight.

Table 7 reports these additional macro results. The macro results cover all 1,000 tables, using the same end-to-end outputs as Table 3. We compute each table's F_1 before averaging, rather than deriving F_1 from macro precision and recall. A completed table with no required changes and no scored edits receives full credit; failed or incomplete executions receive zero. We then average each run's macro scores over the same evaluation runs as Table 3.

Table 7: Table-level macro end-to-end correction results (%). Overall macro P/R/F_1 and macro recall by error type average over all 1,000 tables, using the same runs as Table 3.

P/R/F_1 (%) 0 50 100

Method	Overall macro (%)			Spelling	Formatting	Inconsistency
	P ↑	R ↑	F_1 ↑	R ↑	R ↑	R ↑
Constraint-based data cleaning						
BigDansing	73.12	73.11	73.12	77.9	80.3	82.2
Holistic	72.92	72.91	72.92	77.6	79.9	81.8
DaisyP	79.50	79.50	79.50	84.7	87.2	89.0
Horizon	59.20	59.11	59.13	63.1	65.4	66.6
Relative Trust	50.17	50.17	50.17	52.6	54.4	55.8
NADEEF	73.00	72.91	72.92	77.6	79.9	81.8
LLUNATIC	72.40	72.35	72.33	77.0	79.3	81.3
UniClean	72.60	72.51	72.52	77.1	79.1	81.5
Knowledge-based data cleaning						
KATARA (KB)	67.90	67.90	67.90	72.6	75.0	76.5
Spelling correction methods						
SymSpell	59.56	62.03	59.81	65.4	60.7	60.6
NeuSpell	38.98	41.15	39.20	42.0	40.5	39.5
LLM-prompt cleaning						
CleanAgent	48.72	48.67	48.63	52.0	53.8	56.4
Cocoon	66.21	67.45	66.28	69.7	69.1	71.1
LLM-agent optimized with feedback						
GPT-4.1	75.24	73.48	73.68	76.2	74.4	76.0
GPT-5.2	56.62	56.77	56.15	57.1	55.5	55.6
GPT-5.5	86.21	85.51	85.36	86.6	85.7	85.9
GPT-6-Astra	90.18	90.76	90.03	91.7	91.7	90.5
Coding agents						
GPT-6-Astra (Codex)	86.72	88.31	86.94	88.0	88.3	87.2
GPT-6-Astra (Copilot)	87.58	89.11	87.78	88.9	89.2	88.2
Commercial systems						
Microsoft Excel Clean Data	87.84	85.77	86.25	90.6	90.4	94.0
Google Sheets Smart Cleanup	74.30	74.30	74.30	79.4	82.0	83.2